

R Course: Beginner to Expert

Sri Ram

2025-11-20

Table of contents

1	Welcome	8
2	About this Course	9
2.1	How to Use	9
3	R Basics	10
4	R as a Calculator	11
5	Objects & Assignment	12
6	object naming rules	13
7	Basic Operations in R	14
8	Comments in R	15
9	Getting Help in R	16
10	Installing and Loading Packages in R	17
10.0.1	Saving and Loading Workspaces in R	17
11	Working Directory	18
12	Vectors (Atomic)	19
13	Exercises	20
14	Data Types & Data Structures	21
15	Vectors	22
16	Lists	29
17	Matrices	38
18	Data Frames	52

19 Factors	57
20 Arrays	58
21 Summary	59
22 Manipulating Vectors, Data Frames, and Lists	60
23 Vectors: Indexing & Vectorized Ops	61
24 Data Frames with dplyr	71
25 Read sas dataset	77
26 keeping selected columns	78
27 dropping columns	79
28 Rename	80
29 Arrange (sorting)	81
30 convert var names to lower case	82
31 Creating new variables	83
32 bind rows (set operator in SAS)	84
33 recode (similar to PROC FORMAT in SAS)	85
34 joins	86
34.1 inner join	86
34.2 left join	87
34.3 right join	87
34.4 Full Join	87
34.5 Semi Join	88
34.6 Anti join	88
35 Reshaping data with tidyr	89
36 counting with n()	90
37 dplyr distinct()	91
38 case_when()	92

39 create seq no.	93
40 Lists: lapply, purrr	94
41 Exercises	95
42 String Functions	96
43 String Functions in Base R	97
43.1 Basic String Functions	97
43.2 Pattern Matching and Replacement	97
43.3 String Splitting and Joining	98
43.4 Regular Expressions	98
43.5 Example Usage	99
44 Overall differences	100
45 Detect matches	102
45.1 <code>str_detect()</code> : Detect the presence or absence of a pattern in a string	102
45.2 <code>str_which()</code> : Find positions matching a pattern	102
45.3 <code>str_count()</code> : Count the number of matches in a string	103
45.4 <code>str_locate()</code> : Locate the position of patterns in a string	103
45.5 <code>str_subset()</code> : Keep strings matching a pattern, or find positions	104
45.6 <code>str_extract()</code> : Extract matching patterns from a string	104
45.7 <code>str_match()</code> : Extract matched groups from a string	105
46 Manage lengths	107
46.1 <code>str_length()</code> : The length of a string	107
46.2 <code>str_pad()</code> : Pad a string	108
46.3 <code>str_trunc()</code> : Truncate a character string	108
46.4 <code>str_trim()</code> : Trim whitespace from a string	108
46.5 <code>str_wrap()</code> : Wrap strings into nicely formatted paragraphs	109
47 Mutate strings	110
47.1 <code>str_replace()</code> : Replace matched patterns in a string	110
47.2 <code>case</code> : Convert case of a string	110
48 Join and split	112
48.1 <code>str_flatten()</code> : Flatten a string	112
48.2 <code>str_dup()</code> : duplicate strings within a character vector	112
48.3 <code>str_split()</code> : Split up a string into pieces	113
48.4 <code>str_glue()</code> : Interpolate strings	113

49 Order strings	115
49.1 <code>str_order()</code> : Order or sort a character vector	115
50 date and time functions	117
50.0.1 Date and Time Base Functions in R	117
50.0.2 Creating Date and Time Objects	117
50.0.3 Year Formats	117
50.0.4 Month Formats	117
50.0.5 Day Formats	117
50.0.6 Weekday Formats	118
50.0.7 Time Formats	118
50.1 1. Date & Time Basics in R	120
50.1.1 1.1 Core classes	120
50.1.2 1.2 Converting numerics to Date (origins matter)	120
50.1.3 1.3 Custom date formats	120
50.1.4 1.4 Datetime strings (ISO 8601 & time zones)	121
50.2 2. Lubridate: Fast, Friendly Parsing & Arithmetic	121
50.2.1 2.1 Intuitive parsers	121
50.2.2 2.2 Accessors and rounding	122
50.2.3 2.3 Timespans	122
50.2.4 3 ADaM-Style Partial Date Imputation (Start & End)	122
50.2.5 Start date (AESTDTC)	122
50.2.6 End date (AEENDTC)	123
50.2.7 Censoring by death / last alive date	123
50.2.8 Imputation flags	123
50.3 4. Durations & Analysis Variables	123
50.3.1 4.1 AE duration (AEDUR) in days	123
51 Using <code>difftime</code> (base) or <code>as.numeric</code> on Date differences	124
51.0.1 6.2 Handy lubridate parsers (subset)	124
52 Reading SAS Datasets (+ Cleaning)	125
52.1 Handling Labels & Missing	127
52.2 Labelled to Factor (if needed)	127
52.3 Common Cleaning	127
53 Exercises	128
54 Base R Functions & Apply Family	129
55 Common Utilities	130
56 Apply Family	131

57 Subsetting Essentials	132
58 Exercises	133
59 Custom Functions & Validation	134
60 Writing Functions	135
61 Error Handling	136
62 Unit Testing with testthat	137
63 Document with roxygen2	138
64 Exercises	139
65 R Package Development	140
65.1 Setup	140
65.2 Create a Package	140
65.3 Add a Function	140
65.4 Build, Install, Check	140
65.5 Vignette & Website	140
66 Git in RStudio (Setup & Auth)	141
66.1 One-Time Setup	141
66.2 Initialize Git for the Current Project	141
66.3 Connect to GitHub	141
66.4 Typical Workflow	141
66.5 Remove Git from a Project (macOS/RStudio)	141
67 Creating ADaM: ADSL from SDTM-like Inputs	142
67.1 Build ADSL	142
68 TLFs: Table, Figure, Listing	144
68.1 Table 1: Baseline Characteristics by Treatment	144
68.2 Listing: Subject-Level Listing	145
69 Capstone: End-to-End Mini Workflow	147
69.1 Parameters	147
69.2 1) Read (or Synthesize) SDTM	147
69.3 2) Derive ADSL (Minimal Demo)	148
69.4 3) TLFs	148
69.5 4) Save Outputs	148

70 Appendix: Tips, Profiles, .libPaths	149
70.1 Useful Profiles	149
70.2 Custom Library Paths	149
70.3 Format vs formatC (quick recap)	149
70.4 POSIXct vs POSIXlt	149
70.5 Recommended Packages	150
70.6 Short Glossary	150

1 Welcome

2 About this Course

This Course from **R beginner** to **expert**, with a practical focus on **clinical programming** (CDISC/ADaM and TLFs).

Each chapter includes step-by-step explanations, runnable code, and short exercises.

2.1 How to Use

1. Install: R (4.2), RStudio (or VS Code).
2. Follow along chapter-by-chapter, running code and completing exercises.
3. Use sample data or your own clinical trial datasets (SAS/CSV). ## Structure (Highlights)
 - **Basics & Data:** R syntax, data types/structures, vectors/data frames/lists.
 - **I/O:** Read SAS datasets (with **haven**), handle labels, and clean raw data.
 - **Programming:** Base functions, write your own functions, validate with tests.
 - **DevOps:** Create an R package, connect Git in RStudio/GitHub.
 - **CDISC:** Build ADaM (ADSL) from SDTM-like inputs.
 - **TLFs:** Produce a baseline Table 1, a KM plot, and a listing.

Tip: If you don't have sample SDTM/ADaM data yet, the chapters generate **small synthetic data** as a fallback so everything runs end-to-end. ## contact For questions or feedback, reach out to **r2sas2025@gmail.com**

3 R Basics

4 R as a Calculator

```
[1] 2
```

```
[1] 27
```

5 Objects & Assignment

[1] 13.5

6 object naming rules

- R variable names can contain letters, numbers, periods, and underscores. However, they cannot start with a number or underscore. R is case-sensitive, so `age`, `Age`, and `AGE` would be considered different variables.
- R variable names should be descriptive and meaningful. Avoid using reserved words or function names as variable names.
- A variable can have a short name (like `x` and `y`) or a more descriptive name (`age`, `carname`, `total_volume`). Rules for R variables are:
- A variable name must start with a letter and can be a combination of letters, digits, period(`.`) and underscore(`_`). If it starts with period(`.`), it cannot be followed by a digit.
- A variable name cannot start with a number or underscore (`_`) Variable names are case-sensitive (`age`, `Age` and `AGE` are three different variables) Reserved words cannot be used as variables (`TRUE`, `FALSE`, `NULL`, `if...`)
- Variable names should not contain spaces. Use underscore (`_`) or period (`.`) to separate words in a variable name.
- Variable names should be meaningful and descriptive. Avoid using single-letter variable names except for temporary variables in loops or functions.

7 Basic Operations in R

R supports various basic operations, including:

- * Arithmetic Operations: Addition (+), subtraction (-), multiplication (*), division (/), and exponentiation (^). Example:

```
[1] 15
```

```
[1] 5
```

```
[1] 50
```

```
[1] 2
```

```
[1] 1e+05
```

- Comparison Operations: Equal to (==), not equal to (!=), greater than (>), less than (<), greater than or equal to (>=), and less than or equal to (<=). Example:

```
[1] FALSE
```

```
[1] TRUE
```

```
[1] TRUE
```

```
[1] FALSE
```

```
[1] TRUE
```

```
[1] FALSE
```

- Logical Operations: AND (&), OR (|), and NOT (!). Example:

```
[1] FALSE
```

```
[1] TRUE
```

```
[1] FALSE
```

8 Comments in R

Comments in R are created using the `#` symbol. Anything following the `#` on the same line is considered a comment and is ignored by R during execution. Example:

```
[1] 15
```

9 Getting Help in R

R provides several ways to get help and documentation for functions and packages: *

- `?function_name`: Displays the documentation for a specific function. Example:

- `help(function_name)`: Another way to access the documentation for a function. Example:
- `help.search("keyword")`: Searches for help topics related to a specific keyword. Example:
- `example(function_name)`: Shows examples of how to use a specific function. Example:

```
mean> x <- c(0:10, 50)
```

```
mean> xm <- mean(x)
```

```
mean> c(xm, mean(x, trim = 0.10))  
[1] 8.75 5.50
```

- `vignette("package_name")`: Opens the vignette (detailed documentation) for a specific package. Example:

```
starting httpd help server ... done
```

- `??keyword`: Searches for help topics related to a specific keyword (similar to `help.search`). Example:

10 Installing and Loading Packages in R

R has a vast ecosystem of packages that extend its functionality. To use a package, you need to install it first and then load it into your R session. * Installing a Package: Use the `install.packages("package_name")` function to install a package from CRAN. Example:

- Loading a Package: Use the `library(package_name)` function to load an installed package into your R session. Example:

10.0.1 Saving and Loading Workspaces in R

You can save your R workspace (all objects in memory) to a file and load it later * Saving Workspace: Use the `save.image("file_name.RData")` function to save the entire workspace to a file. Example:

- Loading Workspace: Use the `load("file_name.RData")` function to load a saved workspace from a file. Example:

11 Working Directory

```
[1] "/home/runner/work/r4sas/r4sas"
```

12 Vectors (Atomic)

```
[1] "double"
```

```
[1] "character"
```

```
[1] "logical"
```

13 Exercises

1. Create an object `z` that stores $(5^2 + 7)/3$.
2. Use `?seq` and create a sequence from 0 to 1 by 0.1.
3. Inspect `typeof()` for a few objects you create.

14 Data Types & Data Structures

R has several built-in data structures to store and manipulate different types of data. These include vectors, lists, matrices, data frames, and factors. Below is an overview of each structure along with code examples.

15 Vectors

Vectors are the simplest data structure in R. They store elements of the same type (numeric, character, logical, etc.).

```
[1] 1
```

```
[1] TRUE FALSE TRUE
```

```
[1] TRUE
```

```
[1] TRUE
```

```
[1] "logical"
```

```
[1] 1 2 5
```

```
[1] TRUE
```

```
[1] TRUE
```

```
[1] "integer"
```

```
[1] 1.3 2.1 5.0
```

```
[1] TRUE
```

```
[1] TRUE
```

```
[1] "double"
```

```
[1] "a" "b" "c"
```

[1] TRUE

[1] TRUE

[1] "character"

NULL

[1] "NULL"

[1] "TRUE" "1" "1.25" "a"

[1] TRUE

[1] "character"

[1] 5

[1] "double"

[1] "numeric"

NULL

first	second	third	fourth	fifth
1	2	3	4	5

[1] "first" "second" "third" "fourth" "fifth"

[1]	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
[19]	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36
[37]	37	38	39	40	41	42	43	44	45	46	47	48	49	50	51	52	53	54
[55]	55	56	57	58	59	60	61	62	63	64	65	66	67	68	69	70	71	72
[73]	73	74	75	76	77	78	79	80	81	82	83	84	85	86	87	88	89	90
[91]	91	92	93	94	95	96	97	98	99	100								

```
[1] 10  9  8  7  6  5  4  3  2  1  0 -1 -2 -3 -4 -5 -6 -
7 -8
[20] -9 -10
```

```
[1]  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18
[19] 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36
[37] 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54
[55] 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72
[73] 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88 89 90
[91] 91 92 93 94 95 96 97 98 99 100
```

```
[1] 0.00 0.01 0.02 0.03 0.04 0.05 0.06 0.07 0.08 0.09 0.10 0.11 0.12 0.13 0.14
[16] 0.15 0.16 0.17 0.18 0.19 0.20 0.21 0.22 0.23 0.24 0.25 0.26 0.27 0.28 0.29
[31] 0.30 0.31 0.32 0.33 0.34 0.35 0.36 0.37 0.38 0.39 0.40 0.41 0.42 0.43 0.44
[46] 0.45 0.46 0.47 0.48 0.49 0.50 0.51 0.52 0.53 0.54 0.55 0.56 0.57 0.58 0.59
[61] 0.60 0.61 0.62 0.63 0.64 0.65 0.66 0.67 0.68 0.69 0.70 0.71 0.72 0.73 0.74
[76] 0.75 0.76 0.77 0.78 0.79 0.80 0.81 0.82 0.83 0.84 0.85 0.86 0.87 0.88 0.89
[91] 0.90 0.91 0.92 0.93 0.94 0.95 0.96 0.97 0.98 0.99 1.00
```

```
[1] 0.0 2.5 5.0 7.5 10.0
```

```
a b c d e f g h i j
1 2 3 4 5 6 7 8 9 10
```

```
a e j
1 5 10
```

```
a b c d e
1 2 3 4 5
```

```
a c e g i
1 3 5 7 9
```

```
j i h g f e d c b a
10 9 8 7 6 5 4 3 2 1
```

```
j a e c
10 1 5 3
```


a
1

h i j
8 9 10

a c e
1 3 5

a	b	c	d	e	f	g	h	i	j
1	2	3	4	5	6	7	8	9	10

a	b	c	d	e	f	g	h	i	j
1	2	3	4	5	6	7	8	9	10

a	b	c	d	e	f	g	h	i	j
1	20	3	4	5	6	7	8	9	10

a	b	c	d	e	f	g	h	i	j
0	20	3	4	0	6	7	8	9	0

a	b	c	d	e	f	g	h	i	j
0	20	3	4	0	6	7	8	9	0

a e j
0 0 0

a	b	c	d	e	f	g	h	i	j
1000	20	3	4	1000	6	7	8	9	1000

a	b	c
1000	20	3

a	b	c
1000	20	3

a	b	c
[1,]	1000	20 3

NULL

[1] 1 2 3

[1] 100 101 102 103 104 105

[1] 1 2 3 100 101 102 103 104 105

[1] 1 2 3 1 2 3

[1] 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1
[38] 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2
[75] 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3
[112] 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1
[149] 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2
[186] 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3
[223] 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1
[260] 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2
[297] 3 1 2 3

[1] 10 10 10 10 10 10 10 10 10 10

[1] 1 2 3 4 5

[1] 10

[1] 11 12 13 14 15

[1] -9 -8 -7 -6 -5

[1] 10 20 30 40 50

[1] 0.1 0.2 0.3 0.4 0.5

[1] 1 1024 59049 1048576 9765625

[1] 1 0 1 0 1

[1] 0 1 1 2 2

[1] 1.000000 1.414214 1.732051 2.000000 2.236068

[1] 0.0000000 0.6931472 1.0986123 1.3862944 1.6094379

[1] 15

[1] 13 23 33

[1] 7 17 27

[1] 30 60 90

[1] 3.333333 6.666667 10.000000

[1] 1000 8000 27000

[1] 1 2 0

[1] 3 6 10

[1] 10 20 30

[1] 3 3 3

[1] 180

[,1]
[1,] 180

[,1]
[1,] 180

	[,1]	[,2]	[,3]
[1,]	30	30	30
[2,]	60	60	60
[3,]	90	90	90

```
[1] 1 1 1
```

```
[1] 1 2 3 4 5 6
```

```
[1] 2 3 4 5 6 7
```

```
[1] "a" "b" "c"
```

```
[1] "c" "d" "e"
```

```
[1] "a" "b" "c" "d" "e"
```

```
[1] "c"
```

```
[1] "a" "b"
```

```
[1] FALSE
```

```
[1] TRUE
```

16 Lists

A list can contain elements of different types, including other lists or vectors or data structures.

```
[1] "John"
```

```
[[1]]  
[1] 10
```

```
[[2]]  
[1] 2
```

```
[[3]]  
[1] TRUE
```

```
[[4]]  
[1] "word"
```

```
[[5]]  
  i1  i2  i3  i4  i5  i6  i7  i8  i9 i10  
   1   2   3   4   5   6   7   8   9  10
```

```
[[6]]  
      c1 c2  
r1 10 30  
r2 20 40
```

```
[[7]]  
, , M1
```

```
      c1 c2  
r1  1  3  
r2  2  4
```

```
, , M2
```

```

      c1 c2
r1  5  7
r2  6  8

```

```

List of 7
 $ : num 10
 $ : int 2
 $ : logi TRUE
 $ : chr "word"
 $ : Named int [1:10] 1 2 3 4 5 6 7 8 9 10
 ..- attr(*, "names")= chr [1:10] "i1" "i2" "i3" "i4" ...
 $ : num [1:2, 1:2] 10 20 30 40
 ..- attr(*, "dimnames")=List of 2
 .. ..$ : chr [1:2] "r1" "r2"
 .. ..$ : chr [1:2] "c1" "c2"
 $ : int [1:2, 1:2, 1:2] 1 2 3 4 5 6 7 8
 ..- attr(*, "dimnames")=List of 3
 .. ..$ : chr [1:2] "r1" "r2"
 .. ..$ : chr [1:2] "c1" "c2"
 .. ..$ : chr [1:2] "M1" "M2"

```

```
[1] "list"
```

```
[1] "list"
```

```
[1] TRUE
```

```

$a
[1] 10

```

```

$b
[1] 2

```

```

$c
[1] TRUE

```

```

$d
[1] "word"

```

```

$v
 i1 i2 i3 i4 i5 i6 i7 i8 i9 i10

```

```

1  2  3  4  5  6  7  8  9 10

$M
      c1 c2
r1 10 30
r2 20 40

$A
, , M1

      c1 c2
r1  1  3
r2  2  4

, , M2

      c1 c2
r1  5  7
r2  6  8

$a
[1] 10

$b
[1] 2

$c
[1] TRUE

$d
[1] "word"

$v
  i1  i2  i3  i4  i5  i6  i7  i8  i9 i10
   1   2   3   4   5   6   7   8   9  10

$M
      c1 c2
r1 10 30
r2 20 40

$A

```

```
, , M1
```

```
      c1 c2  
r1  1  3  
r2  2  4
```

```
, , M2
```

```
      c1 c2  
r1  5  7  
r2  6  8
```

```
List of 1  
 $ a: num 10
```

```
[1] "list"
```

```
$a  
[1] 10
```

```
$b  
[1] 2
```

```
$c  
[1] TRUE
```

```
[1] "list"
```

```
 i1  i2  i3  i4  i5  i6  i7  i8  i9 i10  
  1   2   3   4   5   6   7   8   9  10
```

```
[1] TRUE
```

```
      c1 c2  
r1 10 30  
r2 20 40
```

```
[1] "matrix" "array"
```



```

$a
[1] 10

$b
[1] 2

$c
[1] TRUE

$d
[1] "word"

$v
  i1  i2  i3  i4  i5  i6  i7  i8  i9 i10
  1   2   3   4   5   6   7   8   9  10

$M
      c1 c2
r1 10 30
r2 20 40

$A
, , M1

      c1 c2
r1  1  3
r2  2  4

, , M2

      c1 c2
r1  5  7
r2  6  8

$b
[1] 2

$c
[1] TRUE

$d
[1] "word"

```

```
$v
  i1  i2  i3  i4  i5  i6  i7  i8  i9 i10
    1   2   3   4   5   6   7   8   9  10
```

```
$M
    c1 c2
r1 10 30
r2 20 40
```

```
$A
, , M1
```

```
    c1 c2
r1  1  3
r2  2  4
```

```
, , M2
```

```
    c1 c2
r1  5  7
r2  6  8
```

```
[1] 6
```

```
$b
[1] 2
```

```
$c
[1] TRUE
```

```
$d
[1] "word"
```

```
$v
  i1  i2  i3  i4  i5  i6  i7  i8  i9 i10
    1   2   3   4   5   6   7   8   9  10
```

```
$M
    c1 c2
r1 10 30
r2 20 40
```

\$A

, , M1

```
      c1 c2
r1   1  3
r2   2  4
```

, , M2

```
      c1 c2
r1   5  7
r2   6  8
```

[[7]]

[1] 1000

[[1]]

[1] 500

i1	i2	i3	i4	i5	i6	i7	i8	i9	i10
1	2	3	4	5000	6	7	8	9	10

b	c	d	v.i1	v.i2	v.i3	v.i4	v.i5	v.i6	v.i7	v.i8
"2"	"TRUE"	"word"	"1"	"2"	"3"	"4"	"5000"	"6"	"7"	"8"
v.i9	v.i10	M1	M2	M3	M4	A1	A2	A3	A4	A5
"9"	"10"	"10"	"20"	"30"	"40"	"1"	"2"	"3"	"4"	"5"
A6	A7	A8								
"6"	"7"	"8"	"500"							

[1] TRUE

\$b

[1] 2

\$c

[1] TRUE

\$d

[1] "word"

```

$V
  i1  i2  i3  i4  i5  i6  i7  i8  i9  i10
    1   2   3   4 5000   6   7   8   9   10

$M
  c1 c2
r1 10 30
r2 20 40

$A
, , M1

  c1 c2
r1  1  3
r2  2  4

, , M2

  c1 c2
r1  5  7
r2  6  8

[[7]]
[1] 500

$e11
[1]  1  5 10

$e12
[1] TRUE

List of 9
 $ b : int 2
 $ c : logi TRUE
 $ d : chr "word"
 $ v : Named num [1:10] 1 2 3 4 5000 6 7 8 9 10
 ..- attr(*, "names")= chr [1:10] "i1" "i2" "i3" "i4" ...
 $ M : num [1:2, 1:2] 10 20 30 40
 ..- attr(*, "dimnames")=List of 2
 .. ..$ : chr [1:2] "r1" "r2"

```

```

.. ..$ : chr [1:2] "c1" "c2"
$ A : int [1:2, 1:2, 1:2] 1 2 3 4 5 6 7 8
..- attr(*, "dimnames")=List of 3
.. ..$ : chr [1:2] "r1" "r2"
.. ..$ : chr [1:2] "c1" "c2"
.. ..$ : chr [1:2] "M1" "M2"
$ : num 500
$ el1: num [1:3] 1 5 10
$ el2: logi TRUE

```

```
[1] "b" "c" "d" "v" "M" "A" "" "el1" "el2"
```

```
List of 1
```

```

$ :List of 1
..$ :List of 2
.. ..$ : num 1
.. ..$ : logi [1:3] TRUE FALSE FALSE

```

```

[[1]]
[[1]][[1]]
[1] 1

```

```

[[1]][[2]]
[1] TRUE FALSE FALSE

```

```

[[1]]
[1] 1

```

```

[[2]]
[1] TRUE FALSE FALSE

```

```
[1] 1
```

```
[1] TRUE FALSE FALSE
```

17 Matrices

A matrix is a two-dimensional structure that contains elements of the same type (numeric, character, or logical).

```
[1] 4
```

```
      [,1] [,2] [,3]
[1,]    1    4    7
[2,]    2    5    8
[3,]    3    6    9
```

```
      [,1] [,2] [,3]
[1,]    1    2    3
[2,]    4    5    6
[3,]    7    8    9
```

```
      [,1] [,2] [,3]
[1,]    1    3    5
[2,]    2    4    6
```

```
      [,1] [,2] [,3]
v1      1    2    3
v2      4    5    6
v3      7    8    9
```

```
      v1 v2 v3
[1,]  1  4  7
[2,]  2  5  8
[3,]  3  6  9
```

```
[1] 1 2 3 4 5 6 7 8 9
```

	[,1]	[,2]	[,3]
[1,]	1	4	7
[2,]	2	5	8
[3,]	3	6	9

	c1	c2	c3
r1	1	5	9
r2	2	6	10
r3	3	7	11
r4	4	8	12

```
[1] "r1" "r2" "r3" "r4"
```

```
[1] "c1" "c2" "c3"
```

```
[1] 4 3
```

```
$dim
```

```
[1] 4 3
```

```
$dimnames
```

```
$dimnames[[1]]
```

```
[1] "r1" "r2" "r3" "r4"
```

```
$dimnames[[2]]
```

```
[1] "c1" "c2" "c3"
```

```
$dim
```

```
[1] 4 3
```

```
$dimnames
```

```
$dimnames[[1]]
```

```
[1] "row 1" "row 2" "row 3" "row 4"
```

```
$dimnames[[2]]
```

```
[1] "col 1" "col 2" "col 3"
```

	col 1	col 2	col 3
row 1	1	5	9
row 2	2	6	10
row 3	3	7	11
row 4	4	8	12

```
[1] "matrix" "array"
```

```
[1] "integer"
```

```
[1] TRUE
```

	col 1	col 2	col 3
row 1	1	5	9
row 2	2	6	10
row 3	3	7	11
row 4	4	8	12

```
[1] 10
```

row 1	row 2
9	10

	col 1	col 2	col 3
row 2	2	6	10
row 3	3	7	11

	col 2	col 3
row 1	5	9
row 2	6	10
row 3	7	11
row 4	8	12

	col 1	col 2	col 3
row 1	1	5	9
row 2	2	6	10

	col 1	col 2
row 2	2	6
row 3	3	7

	col 2	col 3
row 1	5	9
row 2	6	10
row 3	7	11

	col 1	col 2	col 3
row 1	1	5	9
row 2	2	6	10
row 3	3	7	11
row 4	4	8	12

	col 1	col 2	col 3
row 1	10	5	9
row 2	2	6	10
row 3	3	7	11
row 4	4	8	12

	col 1	col 2	col 3
row 1	10	5	9
row 2	2	6	20
row 3	3	7	20
row 4	4	8	12

	col 1	col 2	col 3
row 1	10	5	9
row 2	2	6	0
row 3	3	7	0
row 4	4	8	0

	row 1	row 2	row 3	row 4
col 1	10	2	3	4
col 2	5	6	7	8
col 3	9	0	0	0

	col 1	col 2	col 3
row 1	10	5	9
row 2	2	6	0
row 3	3	7	0
row 4	4	8	0

	col 1	col 2	col 3
row 1	10	5	9
row 2	2	6	0
row 3	3	7	0
row 4	4	8	0
	0	0	0

	col 1	col 2	col 3
row 1	10	5	9 0
row 2	2	6	0 0
row 3	3	7	0 0
row 4	4	8	0 0

[1] 4 3

	[,1]	[,2]	[,3]	[,4]
[1,]	10	4	7	0
[2,]	2	5	8	0
[3,]	3	6	9	0

	[,1]	[,2]
[1,]	0	0
[2,]	0	0

	[,1]	[,2]
[1,]	1	1
[2,]	1	1

	[,1]	[,2]
[1,]	0	0
[2,]	0	0
[3,]	1	1
[4,]	1	1

	[,1]	[,2]	[,3]	[,4]
[1,]	0	0	1	1
[2,]	0	0	1	1

	[,1]	[,2]	[,3]	[,4]
[1,]	10	4	7	0
[2,]	2	5	8	0
[3,]	3	6	9	0

	[,1]	[,2]	[,3]	[,4]
[1,]	20	14	17	10
[2,]	12	15	18	10
[3,]	13	16	19	10

	[,1]	[,2]	[,3]	[,4]
[1,]	0	-6	-3	-10
[2,]	-8	-5	-2	-10
[3,]	-7	-4	-1	-10

	[,1]	[,2]	[,3]	[,4]
[1,]	100	40	70	0
[2,]	20	50	80	0
[3,]	30	60	90	0

	[,1]	[,2]	[,3]	[,4]
[1,]	1.0	0.4	0.7	0
[2,]	0.2	0.5	0.8	0
[3,]	0.3	0.6	0.9	0

	[,1]	[,2]	[,3]	[,4]
[1,]	1.0000e+10	1048576	282475249	0
[2,]	1.0240e+03	9765625	1073741824	0
[3,]	5.9049e+04	60466176	3486784401	0

	[,1]	[,2]	[,3]	[,4]
[1,]	0	0	1	0
[2,]	0	1	0	0
[3,]	1	0	1	0

	[,1]	[,2]	[,3]	[,4]
[1,]	5	2	3	0
[2,]	1	2	4	0
[3,]	1	3	4	0

	[,1]	[,2]	[,3]	[,4]
[1,]	3.162278	2.000000	2.645751	0
[2,]	1.414214	2.236068	2.828427	0
[3,]	1.732051	2.449490	3.000000	0

	[,1]	[,2]	[,3]	[,4]
[1,]	2.3025851	1.386294	1.945910	-Inf
[2,]	0.6931472	1.609438	2.079442	-Inf
[3,]	1.0986123	1.791759	2.197225	-Inf

[1] 54

	[,1]	[,2]	[,3]
[1,]	4	5	6
[2,]	7	8	9
[3,]	10	11	12

	[,1]	[,2]	[,3]
[1,]	-2	-1	0
[2,]	1	2	3
[3,]	4	5	6

	[,1]	[,2]	[,3]
[1,]	3	6	9
[2,]	12	15	18
[3,]	21	24	27

	[,1]	[,2]	[,3]
[1,]	0.3333333	0.6666667	1
[2,]	1.3333333	1.6666667	2
[3,]	2.3333333	2.6666667	3

	[,1]	[,2]	[,3]
[1,]	1	8	27
[2,]	64	125	216
[3,]	343	512	729

	[,1]	[,2]	[,3]
[1,]	1	2	0
[2,]	1	2	0
[3,]	1	2	0

	[,1]	[,2]	[,3]
[1,]	0	0	1
[2,]	1	1	2
[3,]	2	2	3

	[,1]	[,2]	[,3]
[1,]	1	2	3
[2,]	4	5	6
[3,]	7	8	9

	[,1]	[,2]	[,3]
[1,]	3	3	3
[2,]	3	3	3
[3,]	3	3	3

	[,1]	[,2]	[,3]
[1,]	36	36	36
[2,]	45	45	45
[3,]	54	54	54

	[,1]	[,2]	[,3]
[1,]	18	18	18
[2,]	45	45	45
[3,]	72	72	72

[1] 1 4 2

[1] 74

	[,1]	[,2]	[,3]
[1,]	-0.45945946	0.1081081	0.31081081
[2,]	0.32432432	-0.1351351	-0.01351351
[3,]	-0.05405405	0.1891892	-0.08108108

eigen() decomposition

\$values

[1] 11.778446+0.0000000i -2.389223+0.7578106i -2.389223-0.7578106i

\$vectors

	[,1]	[,2]	[,3]
[1,]	0.4687233+0i	0.5211486+0.2411697i	0.5211486-0.2411697i
[2,]	0.6544420+0i	-0.6642393+0.0000000i	-0.6642393+0.0000000i
[3,]	0.5932993+0i	0.4573822-0.1408153i	0.4573822+0.1408153i

	[,1]	[,2]	[,3]
[1,]	1	5	3
[2,]	2	4	7
[3,]	4	6	2

[1] 9 13 12

[1] 7 15 12

	[,1]	[,2]
[1,]	3	4
[2,]	1	10

	[,1]	[,2]
[1,]	5	24
[2,]	17	66

	[,1]	[,2]	[,3]
[1,]	1	5	3
[2,]	2	4	7
[3,]	4	6	2

[1] 9 13 12

[1] 7 15 12

[1]	-0.4988881236	1.5547123734	-2.0446027734	3.2782719136	-0.7253914348
[6]	-3.3129243739	-1.6554676747	0.6452709115	0.2323652931	-2.0683771723
[11]	0.2643019744	0.5931690346	0.0212461612	-2.0464072782	-0.6792337423
[16]	1.2274470134	1.2291863742	1.3214889867	-0.1166735670	2.4891171222
[21]	5.7766174949	2.9187345770	-1.4941670154	-1.7998459956	-5.6460024668
[26]	-4.1248140745	-6.2929113661	1.1848467176	2.5672459678	1.8020308401
[31]	0.2055153472	2.2772787368	0.2715818657	2.3422329453	0.0548416542
[36]	0.3182627080	-4.1071792023	-0.8035445566	0.7802292191	-1.7632961835
[41]	-0.0552448124	-1.3913951003	0.6787993074	5.9673814033	0.8097839666
[46]	2.0063594006	0.6001230631	0.2424131827	2.6951777286	0.6476507875
[51]	3.6791303985	1.0843573821	0.4254262756	-0.4340580442	2.2394455601
[56]	1.8775451323	1.4060713252	0.2052443476	-0.0374371494	-0.0157341144
[61]	3.4794282143	-2.0179537756	5.5278477996	2.5657971849	-1.7267373411
[66]	0.6872463817	-1.5610753695	-3.5637918846	-1.0571705912	-0.6177853937
[71]	0.5479629968	-1.9590798437	-0.1205740682	2.9408334630	-3.1972715676
[76]	3.6845254276	0.3856297274	-2.9273235928	0.8884141310	-1.5906128688
[81]	-0.5724526046	1.8014921268	-1.8300233772	-3.7160760113	4.8448030623
[86]	1.5439505966	-2.8640608064	-3.2757104878	-0.0912012966	-1.0897332075
[91]	-2.7342137012	0.0050495134	-1.4432299520	1.4549929995	-1.7627823799
[96]	0.3548224346	-0.4530811278	2.1251210819	1.6525640289	1.6039524106
[101]	0.0386759894	-0.7718923760	-1.3812955442	3.6363505080	-0.8265449975
[106]	-1.1940814527	-0.9175857334	3.0286728229	0.3411147056	-0.7197738790

[111]	-0.4869882346	-2.4234538423	-1.9047591767	1.9644981002	-0.6687089642
[116]	1.7037627659	-1.4398554503	-0.8188325461	2.2202518997	1.3811894805
[121]	1.0100472251	-0.7106968211	-1.4370039756	-2.4256738071	-3.4306973044
[126]	2.8377629868	0.2461647469	2.1651152786	-0.3553960869	0.3309916028
[131]	-1.3390823846	-4.6765962235	0.3828313357	1.7610028636	-2.0177057946
[136]	3.2003858938	-0.0241145599	-0.2609654112	-1.5295409787	3.7850754337
[141]	0.2655253926	1.2228473683	-0.0739632826	3.7464113633	-0.2730703169
[146]	3.9776149358	0.3845111647	-3.0563310240	-2.7560628200	0.3147767971
[151]	1.1410611023	2.5801698397	1.6352049088	2.4195181987	1.0510095223
[156]	1.2548448139	1.9054350797	1.5909481729	-2.7010836198	-1.3439495626
[161]	-0.2153862649	-1.2080970938	1.5354525736	0.6528732599	-0.0643659261
[166]	-0.9034238623	-1.9179491737	-1.5689727515	1.0157602812	1.2500005147
[171]	2.4378899618	4.2695041629	0.2782518970	-2.9814016638	2.7358529423
[176]	3.2204179302	1.2802787756	3.0554480142	-3.7380920034	-0.3248189137
[181]	-1.7098408585	0.1329811990	1.8076907548	-5.0934404145	1.2438468995
[186]	2.2935802530	-0.5950040385	-3.6797598251	0.5919323566	-2.4659639738
[191]	-0.5424619328	-0.3611970437	3.7926960904	-3.6105534411	-1.1234867284
[196]	1.5472055553	-0.3370610511	-1.4382135966	-2.8829748349	-1.3778134607
[201]	0.2388597453	-5.4870935088	4.3812230271	0.0420638960	-1.9455663140
[206]	-2.7116221979	-2.8367589366	-2.6709074689	-2.0283083888	-1.3852788605
[211]	0.2469834223	1.5267484619	-1.0697881345	2.6998059631	2.7663642710
[216]	-1.7397764442	-1.1516171579	-0.1758435470	-0.5624053257	-2.3742671094
[221]	-0.7658972955	4.7645668630	0.9339059844	1.3560339223	1.7045443931
[226]	-1.5873158824	0.5363083561	-0.8037108853	-4.1799815940	-0.7705779572
[231]	-1.7043899157	1.6073152888	1.6421550109	1.8432688957	0.5859411289
[236]	0.7261074188	-0.4740320401	-3.2337713871	-2.5242759295	3.4658042861
[241]	0.8219293512	-2.4898627420	0.1259006213	-2.7595573033	1.3202039171
[246]	2.3670562163	-1.0926407442	-3.2476471826	1.5889078160	-0.6895061651
[251]	-0.3224372663	2.0161174935	0.2054848734	1.6616878085	-0.5673445566
[256]	0.0881580419	-0.1924206747	-2.7507385872	0.2316870834	-1.4135338153
[261]	-1.4270344769	0.8809901705	-0.7791850839	-0.6920128317	0.6400209782
[266]	-1.0026822748	1.7147710640	1.4688595575	0.4243938045	-2.9610188154
[271]	-1.0635320583	0.0712412699	0.3701186556	-0.4542823954	2.3976914876
[276]	-1.5753592801	-1.2790163157	-2.7190528021	-2.7586250351	-1.5509961019
[281]	1.4125812527	0.1467134062	1.3451483893	3.1743277664	1.6911710990
[286]	0.8885155824	-0.9155822768	-2.7616523489	-3.1576432686	3.4442091928
[291]	-0.1763942971	-0.0740707780	0.0406749435	-3.6667860802	-2.9163960637
[296]	-0.3592427501	4.6607189652	0.0657256467	-0.2833435941	-1.2190830622
[301]	-1.9519331190	-0.1576685680	-1.7071829237	3.2440258834	1.4159251251
[306]	-2.2363985031	1.4335944143	-3.1252481874	1.2708415632	-2.0977195462
[311]	-1.4914189949	2.7708339729	1.0590730259	-0.5776888363	1.4660412272
[316]	2.3075426542	1.2530647011	-0.4312782756	1.0896641836	0.7688167067
[321]	2.4982458171	3.8227739352	1.5573776965	1.6519044889	1.1794908486

[326]	0.8917529281	1.1080536752	-1.3031932884	-1.7893356536	1.1589245522
[331]	-1.0093320699	-2.5764262838	-0.6434587747	-2.9038453115	-0.7104700087
[336]	0.5437014739	-1.2206618822	4.3664011783	-0.7626259182	3.0059867935
[341]	-1.4289179135	-0.5530640339	0.7039243260	0.2296314531	-1.1859623146
[346]	0.1861925396	3.0463335784	-0.7207038452	-0.5207813453	0.2374730874
[351]	0.7281902839	1.8804143917	2.3512665751	0.3980625531	-2.4017041233
[356]	-0.7566864950	0.6555033283	-0.4132313697	-1.3914478137	1.3403822276
[361]	1.4033078074	-3.2087059949	-1.6432074379	1.3470918833	-0.9059062176
[366]	0.6000593633	1.8441728518	-0.7593746752	0.6984596444	1.3035043585
[371]	-2.2932742387	1.2918712909	-3.9154061694	-0.8628682995	-1.6657554258
[376]	1.5150444655	1.3629098655	0.5095059403	4.1266898503	2.7312040411
[381]	1.2951808727	1.3660983663	-1.1034442865	0.0101996352	1.5309104819
[386]	-1.4007469991	0.9173509759	0.9753143361	-0.5744595600	0.9816914174
[391]	-2.4230032974	1.3222429149	-0.3656407509	1.5138683853	1.6379170110
[396]	-0.3171070943	2.3404029383	-2.2723824382	0.9511817238	-3.3054678449
[401]	0.1890564203	1.7301645251	4.1106642994	-0.0138653463	-1.6948487004
[406]	-3.1438593145	-1.4461524910	-0.5042431641	1.5167743306	-2.5088605032
[411]	0.7331039243	1.6644724682	2.0572716929	-0.2187284703	4.2533911823
[416]	2.2382329037	-0.0337925983	-1.2200129542	-3.3636502896	-1.5109310828
[421]	-1.4997408183	-1.6074084492	-2.2594150385	-0.5238300837	2.2415314481
[426]	-1.4891600784	-0.4626980175	0.5085484265	-0.9405040130	-1.0827604060
[431]	-0.1521860670	-0.7179119903	-1.0757246337	4.4651542054	-3.6643957232
[436]	4.2676603773	0.1532388987	-1.9571688503	1.5922759432	-2.2297053265
[441]	3.7115351322	-1.8636596079	-1.9906626932	0.0497594717	-1.3764268345
[446]	-1.3274557274	-0.2442013288	2.5807522362	-1.5806169189	-2.6889870922
[451]	-2.4134518103	2.5240612951	-3.5638719286	0.6759336586	0.0076505902
[456]	-1.0723504673	5.0753834807	1.9314854720	0.3294057037	-0.4199814473
[461]	5.8740855413	-0.0485713899	0.8735758236	0.2852246443	1.2737739960
[466]	-2.6540510518	2.1184828037	-1.4713987797	-2.6269446825	-3.3469649436
[471]	-0.6292784734	2.8677755535	-2.8327611718	-4.7238919499	-0.5002734987
[476]	-0.5627776318	2.4750938096	-3.0581891992	1.9989113271	-2.4546135468
[481]	-1.4681333192	-1.5649302893	1.4905786071	0.6747889827	0.6451992271
[486]	-4.1827164218	-1.3421870991	-1.0388841706	-0.0549999747	2.4879117151
[491]	1.4569542196	-0.9823148649	1.7187596438	-1.9833938739	0.8452420370
[496]	-1.4365372796	2.2692316927	1.6378421199	-1.8955985194	-1.6820954535
[501]	1.1721279010	-3.6187407277	1.9604804880	0.0900629380	1.5820697883
[506]	1.1029925806	-1.9088612339	-2.7542154977	-0.6262103930	0.7592658117
[511]	-3.0345095822	-2.7440175947	-0.4376893070	-0.5631419031	0.2286292921
[516]	-0.7416656056	0.4680975496	2.3162175231	-1.2436930166	4.1215881639
[521]	-1.1402593842	4.8980165957	-0.3012329302	-0.9118759795	1.2388585728
[526]	-2.8188618957	-0.0607084104	-2.4971158066	-3.2796523091	2.1425390190
[531]	0.9591014261	-0.2159605517	0.3927911058	0.9877115504	2.3783097864
[536]	-0.6395200870	0.4029348799	1.8436038904	1.5372749929	-3.0163284573

[541]	-0.8946021076	-0.1497883989	-1.1461343409	0.1150411056	2.2942016293
[546]	-2.0650729131	-1.1337484898	2.6667587599	3.0845752182	1.4653574929
[551]	-4.3886091878	1.0638809397	-2.6118837933	-0.7190176938	-0.4763614990
[556]	1.6023591622	-0.6424426383	1.3417095031	-1.3548028641	0.9377060057
[561]	-1.1871062011	-1.5384489825	-0.4453862846	1.0891299712	-0.0236166552
[566]	4.9994329928	0.6304247433	0.1845254281	1.1899728180	1.3533396574
[571]	-2.1085515877	4.0733673807	0.4647491200	0.6496128997	-0.0832791252
[576]	2.2204682602	1.9899636522	1.0050746445	0.8988180732	-3.0574369918
[581]	-1.3647866794	0.1689101795	-0.2676101561	-1.0006521013	-0.2988158621
[586]	-0.8970179939	2.7650068356	0.5601389631	-1.1235062726	0.4778371738
[591]	0.0052353400	1.0696303400	2.5137225988	0.9540201871	-0.6058611898
[596]	1.1100547749	-0.6129241312	-0.7143176257	-0.1712512650	-0.5396735909
[601]	1.9788111250	-3.9842178151	0.4822234357	-2.7791715409	-0.4971864776
[606]	1.5037073858	2.2206756959	0.1956620958	-1.1989727443	1.5608064809
[611]	0.0072156276	-1.0230085566	2.0168261561	-2.6042897733	2.9014560739
[616]	1.2667165574	0.2895273023	0.0559154584	2.2612289674	5.5900949087
[621]	1.2438293421	4.3260270944	-2.6742190980	3.6551738333	0.0837044797
[626]	-1.0646941874	1.0234012399	1.0101007573	0.7975563219	2.9290615924
[631]	-5.4928549175	-1.2076699876	1.4015496126	1.9534330082	2.7121795685
[636]	0.9363517182	1.9548311745	0.1899952540	-1.4869500981	-3.7003297027
[641]	-2.5360473337	-3.3011307477	-0.1840541453	-1.1923034137	-2.0762815458
[646]	3.2741045283	2.4591241559	-0.6592092290	-0.3055262855	0.3740312500
[651]	-1.2355060745	-0.0004077603	1.0739978211	-0.3576568689	0.6487275096
[656]	0.0493854887	-0.2048665263	-0.7765010259	-2.2079863991	0.6207242302
[661]	2.3800157259	-0.6467971041	-4.0837293385	-1.0147818377	-2.3243317623
[666]	0.2528207710	-1.3443728707	-1.0359515391	-2.2067008144	-0.2857672930
[671]	-1.8335457707	-1.2340299070	0.5661598576	3.4860664337	2.5070598867
[676]	1.7849286803	0.6649256357	-3.5871673478	-1.0499825011	1.6467807156
[681]	0.9092834960	2.5234071314	2.3757152436	1.0130665306	-2.1929685658
[686]	-1.5759977530	-1.3945458008	0.0113475725	0.8336731541	-3.7219214373
[691]	-0.8587535111	1.2612259726	-0.5198435043	0.3350506486	-2.5176380382
[696]	0.5413374389	2.1227005126	0.9308667032	2.0541972375	2.1793820604
[701]	-1.7184165822	-0.7792626815	1.3279870165	1.9930391884	-0.2275703965
[706]	1.2390821714	-1.3896467030	2.7882327154	0.0470057958	-0.7295559722
[711]	1.8780972610	2.4379156532	3.0931615523	-0.0035622766	-2.6812748595
[716]	2.0873301792	-3.1372087994	3.4116829361	0.4137526706	1.7721296415
[721]	3.0402914480	3.8642731270	-2.3797660447	-0.9148346191	1.7850992067
[726]	-0.4071321317	-0.8522320208	-1.8444108735	1.5494349320	-5.0265538350
[731]	0.2897792060	0.4145275086	-2.0783708555	0.5267635492	2.1784563174
[736]	-1.5085686127	3.1836772852	-0.7634861736	-0.7921542808	0.6390728581
[741]	-1.0072147147	1.8504056914	1.0149445665	-2.5779466165	0.0642436815
[746]	-1.8568701097	-0.6161436198	-2.1741273744	2.9976697853	-1.2990938769
[751]	-0.3334686697	1.1262097961	-0.5459240582	-1.1166486031	1.1297193366

[756]	2.0617081807	0.8009197115	0.1502478387	2.0517523307	1.0889810170
[761]	-2.1015612128	-0.1811168499	1.1439706600	-1.3587013848	0.6980554225
[766]	-0.7458847598	-0.8835595107	0.7458046214	-0.1533278145	-2.1179568633
[771]	0.1693608037	-1.4885252033	-2.2346074153	1.5653645065	0.6813315019
[776]	-2.5043330682	1.1428656318	1.4245891083	1.8155402167	2.6554198022
[781]	2.1736387630	4.1247832962	2.0270589977	-2.5000634204	1.5366278843
[786]	2.0455682739	-1.3986040943	0.9458935709	-0.8942087134	-1.1199297525
[791]	0.1520112712	0.6531492896	-0.1614435249	-4.2572682535	1.4531543105
[796]	0.4599070514	0.2224244403	1.0733781415	2.2254557104	-0.6115893663
[801]	-0.3672661740	-2.1674852670	2.6125567322	1.2389344446	4.1577514759
[806]	3.8324682535	-2.0645662012	-6.3529595359	-0.9574620508	0.0409985164
[811]	3.5863943368	2.3802680051	-0.3695645517	0.6135255720	-1.5224300358
[816]	-0.8027390090	-0.1790624909	0.3842500498	1.8933273319	-3.2739335446
[821]	0.0713359562	0.7238422266	2.4317124673	-1.3621980670	-0.3406895281
[826]	1.1253800492	-1.5849756940	-2.3056647186	1.1386232369	3.3125985628
[831]	1.7280474560	-1.0625073722	1.5484318987	-0.1383582251	1.2130469468
[836]	-0.0177700241	-4.4894729020	1.1438599108	0.4060771535	1.9486658526
[841]	1.5340115101	-0.1188030515	-0.1034681900	-1.5714682094	-0.5750094143
[846]	-1.0579308234	3.4912937411	0.4032424225	1.7010375684	-1.8397506710
[851]	1.8289208964	-0.0025649423	1.4207935899	1.4986087549	-1.3859021285
[856]	-3.3422028985	4.5578440212	-1.7402743010	-0.2309798285	4.0267334957
[861]	-3.6896742374	1.3627849141	-0.4563322877	-1.8190717327	-1.3102718682
[866]	0.9455480005	-0.5086386661	3.7399040945	2.0554810455	-0.4033984324
[871]	0.7781673622	3.3646210550	2.1922564786	-1.2599134369	-0.5075916134
[876]	-1.5665648029	2.4262038572	-1.5337435787	1.5429687088	-1.3386374680
[881]	-0.5856142835	-0.5950331621	-0.8479146805	2.2794731876	-2.4634807224
[886]	-0.7598478889	-1.4721480609	-0.1222687131	2.6668603875	-1.1011528498
[891]	-2.9016694046	3.7977454169	1.3260140285	2.8176283104	-1.8444955899
[896]	0.4843465851	-2.9885907705	1.0323522086	2.6722749215	-0.7374600697
[901]	0.7911980727	-1.0157052863	-1.5405150008	-0.4266063038	0.8166244701
[906]	-1.0351941476	-0.4467461983	-2.8385435205	-3.3149845565	-1.5617768792
[911]	-0.3856967473	-1.4952269751	-2.0579468418	0.5247052271	-2.1282138884
[916]	0.2695131021	1.3975251594	-5.0046970546	-1.3524972573	1.2091754805
[921]	-1.6223040669	-4.0660055674	2.4484076598	0.5554394065	1.1620277732
[926]	0.8765259333	6.8303432924	-1.8734509647	1.5597370807	-3.7849316352
[931]	2.6138574262	-0.2832086577	4.6391468291	2.8123297939	1.9194902904
[936]	-2.2598382497	1.6840720383	-3.0081145574	0.3926598718	-0.8338839103
[941]	0.6705133185	0.6961557845	-3.1431926129	-3.3205436495	-1.1947000060
[946]	1.2642306001	-0.2974336200	2.2079479809	-0.9653125592	2.8225771780
[951]	0.2437993046	0.3925125521	-0.0384543419	-1.2965794168	-1.5659433421
[956]	0.2266915346	-1.4185694693	-0.5779090900	-1.5313556633	-2.7098566981
[961]	5.3221159178	-1.4517372195	-0.3112294952	0.7652240295	0.0586330432
[966]	1.1145614935	3.4014332727	1.4319338661	-1.5390437161	-3.3410826455

[971] -2.2135201574 0.9693835304 1.9412222868 -1.8655098055 -1.9711243210
 [976] -0.2860613910 2.8154712335 0.2926782180 0.0460805571 -1.9492246004
 [981] -0.9249523739 2.8638317616 -1.9404314509 1.8320402649 -0.1660319679
 [986] -0.4308084466 0.1756737289 1.0616372711 -2.3019108761 -0.4428725220
 [991] 1.9054586862 4.6464948834 -2.8826312697 -1.2501722071 0.7307405649
 [996] 0.3911475789 2.6730958656 3.2397208566 1.3451069058 -0.6503776127

[1] 0.275145534 0.005352452 -0.149750117 -0.109480763 0.278471873
 [6] -0.130738384 0.023632030 -0.132103174 0.431069363 -0.379496632

[1] -0.151026453 0.178713839 0.473219699 -0.403717738 -1.268839192
 [6] 0.116537229 -0.058804538 0.548855065 1.130542371 1.131906178
 [11] -0.368977925 -0.411788700 0.901640722 0.884500859 -0.540110831
 [16] 0.155434108 -0.339501262 1.370609838 -0.003207332 0.017781394
 [21] 0.361717900 -0.296238702 0.608985491 -0.492292898 -0.484272661
 [26] 0.476809415 -1.813269176 -0.156846749 -0.755696273 0.004703264
 [31] -0.325748385 0.130283275 0.180047778 1.117169318 -0.927892516
 [36] 0.435429231 -0.003682181 0.249940914 -0.793991399 0.535539847
 [41] 0.177658771 0.016758536 0.478505712 0.222469609 -0.581580632
 [46] -0.628067149 -0.636614759 -0.809749773 0.308824566 0.964593901
 [51] 1.465872387 0.580203379 -0.863298282 -1.456985488 0.122061532
 [56] -1.006432305 -0.070716798 -0.650407796 0.209426754 -0.028310954
 [61] 1.324076979 -1.118861608 0.051250106 -0.221849538 1.104469044
 [66] -1.008665081 0.255462354 1.929581441 0.773615901 -0.120494722
 [71] 0.134139473 0.200162384 -0.104633816 -0.191093878 -0.457557852
 [76] 0.996225717 0.492995625 -0.597837492 -0.094015843 -0.405078987
 [81] -0.787646902 -0.348605780 0.707559319 -0.197351905 -0.187796405
 [86] 0.262395843 -1.221067538 -0.305705395 0.480569647 0.230528881
 [91] 0.181443961 -0.105643006 0.265377105 -0.240875364 0.653604493
 [96] -0.002393267 -0.270093179 0.173874083 -0.645591505 0.307874493

[1] 2.174773 2.250221 2.073695 1.819164 2.030336 1.944039 2.169034 2.305846
 [9] 2.028467 1.798903

18 Data Frames

A data frame is a table where each column can contain elements of different types (e.g., numbers, strings). It's the most common structure used for data sets.

```
      Name Age Gender
1   Alice  23      F
2     Bob  30      M
3 Charlie  25      M
```

```
[1] "Alice"  "Bob"    "Charlie"
```

```
'data.frame':  10 obs. of  3 variables:
 $ num      : num  10 20 30 40 50 60 70 80 90 100
 $ date     : Date, format: "2020-01-01" "2020-01-08" ...
 $ string   : chr  "word" "word" "word" "word" ...
```

```
      col1 col2 col3 col4 col5 col6 col7 col8 col9 col10
row1     1     2     3     4     5     6     7     8     9    10
row2    11    12    13    14    15    16    17    18    19    20
row3    21    22    23    24    25    26    27    28    29    30
row4    31    32    33    34    35    36    37    38    39    40
row5    41    42    43    44    45    46    47    48    49    50
row6    51    52    53    54    55    56    57    58    59    60
row7    61    62    63    64    65    66    67    68    69    70
row8    71    72    73    74    75    76    77    78    79    80
row9    81    82    83    84    85    86    87    88    89    90
row10   91    92    93    94    95    96    97    98    99   100
```

```
      col1 col2 col3 col4 col5 col6 col7 col8 col9 col10
row1     1     2     3     4     5     6     7     8     9    10
row2    11    12    13    14    15    16    17    18    19    20
row3    21    22    23    24    25    26    27    28    29    30
row4    31    32    33    34    35    36    37    38    39    40
row5    41    42    43    44    45    46    47    48    49    50
```

row6	51	52	53	54	55	56	57	58	59	60
row7	61	62	63	64	65	66	67	68	69	70
row8	71	72	73	74	75	76	77	78	79	80
row9	81	82	83	84	85	86	87	88	89	90
row10	91	92	93	94	95	96	97	98	99	100

```
[1] 10 10
```

```
[1] 10
```

```
[1] 10
```

```
[1] "data.frame"
```

```
[1] "list"
```

```

      name
1   Max
2  Jane
3  John
4  Tony
5 Janis
6 Helen

```

```
[1] "data.frame"
```

```
[1] 55 35 46 22 60 27
```

```
[1] "numeric"
```

```
[1] 55 35 46 22 60 27
```

```
[1] "numeric"
```

```

      name age
1   Max  55
2  Jane  35
3  John  46
4  Tony  22
5 Janis  60
6 Helen  27

```

	id	name	surname	age	date_start_work	gender	manager_position
1	1	Max	Gordon	55	1985-09-01	M	TRUE
2	2	Jane	Smith	35	2010-10-01	F	FALSE
3	3	John	Don	46	1999-06-01	M	FALSE
4	4	Tony	Price	22	2019-03-01	M	FALSE
5	5	Janis	Jett	60	1980-04-15	F	TRUE
6	6	Helen	Dust	27	2015-02-20	M	FALSE

[1] "Jane"

[1] "Jane"

	gender	manager_position
1	M	TRUE
2	F	FALSE
3	M	FALSE
4	M	FALSE

	gender	manager_position
1	M	TRUE
2	F	FALSE
3	M	FALSE
4	M	FALSE

[1] 1 2 3 4 5 6

[1] 1 2 3 4 5 6

[1] 1 2 3 4 5 6

	id	name	surname	age	date_start_work	gender	manager_position
5	5	Janis	Jett	60	1980-04-15	F	TRUE
6	6	Helen	Dust	27	2015-02-20	M	FALSE

	id	name	surname	age	date_start_work	gender	manager_position
5	5	Janis	Jett	60	1980-04-15	F	TRUE
6	6	Helen	Dust	27	2015-02-20	M	FALSE

id	name	surname	age
Min. :1.0	Length:7	Length:7	Min. :22.00
1st Qu.:2.5	Class :character	Class :character	1st Qu.:29.50
Median :4.0	Mode :character	Mode :character	Median :35.00
Mean :4.0			Mean :39.57
3rd Qu.:5.5			3rd Qu.:50.50
Max. :7.0			Max. :60.00
date_start_work	gender	manager_position	role
Min. :1980-04-15	Length:7	Mode :logical	Length:7
1st Qu.:1992-07-16	Class :character	FALSE:5	Class :character
Median :2010-10-01	Mode :character	TRUE :2	Mode :character
Mean :2004-05-06			
3rd Qu.:2017-02-24			
Max. :2020-01-01			
eye_color	hair_color		
Length:7	Length:7		
Class :character	Class :character		
Mode :character	Mode :character		

id	name	surname	age	date_start_work	gender	manager_position	role
1	1	Max	Gordon	55	1985-09-01	M	TRUE director
3	3	John	Don	46	1999-06-01	M	FALSE analyst
4	4	Tony	Price	22	2019-03-01	M	FALSE researcher
6	6	Helen	Dust	27	2015-02-20	M	FALSE analyst
7	7	Mark	Jax	32	2020-01-01	M	FALSE researcher

eye_color	hair_color
1	blue blonde
3	brown black
4	hazel brown
6	brown dark brown
7	brown brown

id	name	surname	age	date_start_work	gender	manager_position	role	eye_color
5	5	Janis	Jett	60	1980-04-15	F	TRUE CEO	blue
hair_color								
5								blonde

id	name	surname	age	date_start_work	gender	manager_position	role
1	1	Max	Gordon	55	1985-09-01	M	TRUE director

3	3	John	Don	46	1999-06-01	M	FALSE	analyst
4	4	Tony	Price	22	2019-03-01	M	FALSE	researcher
6	6	Helen	Dust	27	2015-02-20	M	FALSE	analyst
7	7	Mark	Jax	32	2020-01-01	M	FALSE	researcher

	eye_color	hair_color
1	blue	blonde
3	brown	black
4	hazel	brown
6	brown	dark brown
7	brown	brown

	id	name	surname	age	date_start_work	gender	manager_position	role	eye_color
5	5	Janis	Jett	60	1980-04-15	F	TRUE	CEO	blue
									hair_color
5									blonde

age	manager_position
277	2

19 Factors

Factors are used to represent categorical data. They store both the data values and the corresponding levels.

20 Arrays

Arrays are similar to matrices but can have more than two dimensions.

```
[1] 4
```

21 Summary

- **Vector:** One-dimensional, homogeneous.
- **List:** One-dimensional, heterogeneous.
- **Matrix:** Two-dimensional, homogeneous.
- **Data Frame:** Two-dimensional, heterogeneous (columns can be different types).
- **Factor:** Categorical data representation.
- **Array:** Multi-dimensional, homogeneous.

22 Manipulating Vectors, Data Frames, and Lists

Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

`filter`, `lag`

The following objects are masked from 'package:base':

`intersect`, `setdiff`, `setequal`, `union`

23 Vectors: Indexing & Vectorized Ops

```
NULL
```

```
[1] "one"  "two"  "three"
```

```
  one  two three  
    1    2    3
```

```
[1] 3
```

```
[1] 1 5
```

```
[1] 1 5 7
```

```
[1] 3 7
```

```
numeric(0)
```

```
[1] "numeric"
```

```
[1] 11 13 15 17
```

```
[1] -4 -2  0  2
```

```
[1] 10 30 50 70
```

```
[1] 0.1 0.3 0.5 0.7
```

```
[1] 0 1 2 3
```

```
[1] 1 1 1 1
```

```
[1] 1
```

```
[1] 7
```

```
[1] 4
```

```
[1] 4
```

```
[1] 1 7
```

```
[1] 6.666667
```

```
[1] 2.581989
```

0%	25%	50%	75%	100%
1.0	2.5	4.0	5.5	7.0

25%	90%
5.75	18.10

```
[1] 3
```

```
[1] FALSE FALSE FALSE FALSE  TRUE  TRUE  TRUE  TRUE
```

```
[1] 2
```

```
[1] 1
```

```
[1] 2
```

```
[1] 1
```

```
[1] 133
```

```
[1] 7
```

```
[1] 133
```

```

[1] 7

[1] 3

[1] 4

[1] 1 2 3 4 5 6 7 8

[1] 8 7 4 5 11 112 133 33

[1] 8

[1] 1 2 3 4 5 6 7 8

[1] 1 2 3 4 5 6 7 8

[1] FALSE TRUE TRUE FALSE

[1] 11 7

[1] 11 7
attr("na.action")
[1] 2 3
attr("class")
[1] "omit"

a b
1 2

      Last      First
"Kumman" "Ramesh"

[1] TRUE

[1] FALSE

[1] FALSE FALSE FALSE FALSE TRUE TRUE TRUE TRUE TRUE TRUE

```

```

[1] TRUE TRUE TRUE TRUE TRUE TRUE FALSE FALSE FALSE FALSE FALSE

[1] FALSE FALSE FALSE FALSE TRUE TRUE TRUE TRUE TRUE TRUE

[1] TRUE TRUE TRUE TRUE TRUE TRUE FALSE FALSE FALSE FALSE FALSE

[1] "string"

[1] "ab"

[1] "i.1" "i.2" "i.3" "i.4" "i.5" "i.6" "i.7" "i.8" "i.9" "i.10"

[1] "i.1" "j.2" "k.3" "i.4" "j.5" "k.6" "i.7" "j.8" "k.9" "i.10"

[1] "i1j2k3"

[1] "i1" "j2" "k3"

[1] "Helloworld,IuseR"

[1] "Hello world , I use R"

Hello world !

Hello/world/!

[1] 11

      Murder Assault UrbanPop Rape
Alabama    13.2    236      58 21.2
Alaska     10.0    263      48 44.5
Arizona     8.1    294      80 31.0
Arkansas    8.8    190      50 19.5
California  9.0    276      91 40.6
Colorado   7.9    204      78 38.7

```


[1]	"Alabama"	"Alaska"	"Arizona"	"Arkansas"
[5]	"California"	"Colorado"	"Connecticut"	"Delaware"
[9]	"Florida"	"Georgia"	"Hawaii"	"Idaho"
[13]	"Illinois"	"Indiana"	"Iowa"	"Kansas"
[17]	"Kentucky"	"Louisiana"	"Maine"	"Maryland"
[21]	"Massachusetts"	"Michigan"	"Minnesota"	"Mississippi"
[25]	"Missouri"	"Montana"	"Nebraska"	"Nevada"
[29]	"New Hampshire"	"New Jersey"	"New Mexico"	"New York"
[33]	"North Carolina"	"North Dakota"	"Ohio"	"Oklahoma"
[37]	"Oregon"	"Pennsylvania"	"Rhode Island"	"South Carolina"
[41]	"South Dakota"	"Tennessee"	"Texas"	"Utah"
[45]	"Vermont"	"Virginia"	"Washington"	"West Virginia"
[49]	"Wisconsin"	"Wyoming"		

[1]	"ALABAMA"	"ALASKA"	"ARIZONA"	"ARKANSAS"
[5]	"CALIFORNIA"	"COLORADO"	"CONNECTICUT"	"DELAWARE"
[9]	"FLORIDA"	"GEORGIA"	"HAWAII"	"IDAHO"
[13]	"ILLINOIS"	"INDIANA"	"IOWA"	"KANSAS"
[17]	"KENTUCKY"	"LOUISIANA"	"MAINE"	"MARYLAND"
[21]	"MASSACHUSETTS"	"MICHIGAN"	"MINNESOTA"	"MISSISSIPPI"
[25]	"MISSOURI"	"MONTANA"	"NEBRASKA"	"NEVADA"
[29]	"NEW HAMPSHIRE"	"NEW JERSEY"	"NEW MEXICO"	"NEW YORK"
[33]	"NORTH CAROLINA"	"NORTH DAKOTA"	"OHIO"	"OKLAHOMA"
[37]	"OREGON"	"PENNSYLVANIA"	"RHODE ISLAND"	"SOUTH CAROLINA"
[41]	"SOUTH DAKOTA"	"TENNESSEE"	"TEXAS"	"UTAH"
[45]	"VERMONT"	"VIRGINIA"	"WASHINGTON"	"WEST VIRGINIA"
[49]	"WISCONSIN"	"WYOMING"		

[1]	"alabama"	"alaska"	"arizona"	"arkansas"
[5]	"california"	"colorado"	"connecticut"	"delaware"
[9]	"florida"	"georgia"	"hawaii"	"idaho"
[13]	"illinois"	"indiana"	"iowa"	"kansas"
[17]	"kentucky"	"louisiana"	"maine"	"maryland"
[21]	"massachusetts"	"michigan"	"minnesota"	"mississippi"
[25]	"missouri"	"montana"	"nebraska"	"nevada"
[29]	"new hampshire"	"new jersey"	"new mexico"	"new york"
[33]	"north carolina"	"north dakota"	"ohio"	"oklahoma"
[37]	"oregon"	"pennsylvania"	"rhode island"	"south carolina"
[41]	"south dakota"	"tennessee"	"texas"	"utah"
[45]	"vermont"	"virginia"	"washington"	"west virginia"
[49]	"wisconsin"	"wyoming"		

[1] "Hello World"

[1]	"Alabama"	"Alaska"	"Arizona"	"Arkansas"
[5]	"California"	"Colorado"	"Connecticut"	"Delaware"
[9]	"Florida"	"Georgia"	"Hawaii"	"Idaho"
[13]	"Illinois"	"Indiana"	"Iowa"	"Kansas"
[17]	"Kentucky"	"Louisiana"	"Maine"	"Maryland"
[21]	"Massachusetts"	"Michigan"	"Minnesota"	"Mississippi"
[25]	"Missouri"	"Montana"	"Nebraska"	"Nevada"
[29]	"New Hampshire"	"New Jersey"	"New Mexico"	"New York"
[33]	"North Carolina"	"North Dakota"	"Ohio"	"Oklahoma"
[37]	"Oregon"	"Pennsylvania"	"Rhode Island"	"South Carolina"
[41]	"South Dakota"	"Tennessee"	"Texas"	"Utah"
[45]	"Vermont"	"Virginia"	"Washington"	"West Virginia"
[49]	"Wisconsin"	"Wyoming"		

[1]	"Wyoming"	"Wisconsin"	"West Virginia"	"Washington"
[5]	"Virginia"	"Vermont"	"Utah"	"Texas"
[9]	"Tennessee"	"South Dakota"	"South Carolina"	"Rhode Island"
[13]	"Pennsylvania"	"Oregon"	"Oklahoma"	"Ohio"
[17]	"North Dakota"	"North Carolina"	"New York"	"New Mexico"
[21]	"New Jersey"	"New Hampshire"	"Nevada"	"Nebraska"
[25]	"Montana"	"Missouri"	"Mississippi"	"Minnesota"
[29]	"Michigan"	"Massachusetts"	"Maryland"	"Maine"
[33]	"Louisiana"	"Kentucky"	"Kansas"	"Iowa"
[37]	"Indiana"	"Illinois"	"Idaho"	"Hawaii"
[41]	"Georgia"	"Florida"	"Delaware"	"Connecticut"
[45]	"Colorado"	"California"	"Arkansas"	"Arizona"
[49]	"Alaska"	"Alabama"		

[1] "Ala"

Loading required package: countrycode

[1] 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16

[1]	"Afghanistan"	"Albania"	"Algeria"
[4]	"American Samoa"	"Andorra"	"Angola"
[7]	"Anguilla"	"Antarctica"	"Antigua & Barbuda"
[10]	"Argentina"	"Armenia"	"Aruba"
[13]	"Australia"	"Austria"	"Austria-Hungary"
[16]	"Azerbaijan"		

[1] "Afghanistan"	"Albania"	"Algeria"
[4] "American Samoa"	"Andorra"	"Angola"
[7] "Anguilla"	"Antarctica"	"Antigua & Barbuda"
[10] "Argentina"	"Armenia"	"Aruba"
[13] "Australia"	"Austria"	"Austria-Hungary"
[16] "Azerbaijan"		

[1] "Austria-Hungary"	"British Indian Ocean Territory"
[3] "Germany"	"Guernsey"
[5] "Hungary"	"Italy"
[7] "Jersey"	"Norway"
[9] "Paraguay"	"Saxony"
[11] "St. Barthélemy"	"Turkey"
[13] "Tuscany"	"Uruguay"
[15] "Vatican City"	

- [1] "American Samoa"
- [2] "Bouvet Island"
- [3] "British Indian Ocean Territory"
- [4] "British Virgin Islands"
- [5] "Burkina Faso"
- [6] "Cape Verde"
- [7] "Caribbean Netherlands"
- [8] "Cayman Islands"
- [9] "Central African Republic"
- [10] "Channel Islands"
- [11] "Christmas Island"
- [12] "Cook Islands"
- [13] "Costa Rica"
- [14] "Côte d'Ivoire"
- [15] "Dominican Republic"
- [16] "El Salvador"
- [17] "Equatorial Guinea"
- [18] "Falkland Islands"
- [19] "Faroe Islands"
- [20] "French Guiana"
- [21] "French Polynesia"
- [22] "French Southern Territories"
- [23] "German Democratic Republic"
- [24] "Heard & McDonald Islands"
- [25] "Hesse Electoral"
- [26] "Hesse Grand Ducal"

[27] "Hong Kong SAR China"
[28] "Isle of Man"
[29] "Macao SAR China"
[30] "Marshall Islands"
[31] "Mecklenburg Schwerin"
[32] "Micronesia (Federated States of)"
[33] "Netherlands Antilles"
[34] "New Caledonia"
[35] "New Zealand"
[36] "Norfolk Island"
[37] "North Korea"
[38] "North Macedonia"
[39] "Northern Mariana Islands"
[40] "Orange Free State"
[41] "Palestinian Territories"
[42] "Papua New Guinea"
[43] "Pitcairn Islands"
[44] "Puerto Rico"
[45] "Republic of Vietnam"
[46] "Saint Martin (French part)"
[47] "San Marino"
[48] "Saudi Arabia"
[49] "Serbia and Montenegro"
[50] "Sierra Leone"
[51] "Sint Maarten"
[52] "Solomon Islands"
[53] "South Africa"
[54] "South Georgia & South Sandwich Islands"
[55] "South Korea"
[56] "South Sudan"
[57] "Sri Lanka"
[58] "Svalbard & Jan Mayen"
[59] "São Tomé & Príncipe"
[60] "Turks & Caicos Islands"
[61] "Two Sicilies"
[62] "U.S. Virgin Islands"
[63] "United Arab Emirates"
[64] "United Arab Republic"
[65] "United Kingdom"
[66] "United Province CA"
[67] "United States"
[68] "United States Minor Outlying Islands (the)"
[69] "Vatican City"

[70] "Western Sahara"
 [71] "Yemen Arab Republic"
 [72] "Yemen People's Republic"
 [73] "Åland Islands"

[1] "Belize"	"Brunei"	"Burundi"
[4] "Cape Verde"	"Chile"	"Congo - Brazzaville"
[7] "Côte d'Ivoire"	"Djibouti"	"Eswatini"
[10] "Fiji"	"France"	"Greece"
[13] "Guadeloupe"	"Haiti"	"Kiribati"
[16] "Malawi"	"Mali"	"Martinique"
[19] "Mayotte"	"Mozambique"	"Niue"
[22] "Orange Free State"	"Sierra Leone"	"Singapore"
[25] "Suriname"	"São Tomé & Príncipe"	"Timor-Leste"
[28] "Ukraine"	"Zimbabwe"	

[1] "British Virgin Islands" "U.S. Virgin Islands"

[1] "dollar \$" "dollar" "US dollar"

[1] "dollar \$" "dollar" "US dollar"

[1] "word.word"

[1] "Word"

[1] "worD"

[1] "123" "1twothree"

[1] "onetwothree" "1twothree"

[1] "dollar"

[1] "123"

[1] "I have started to learn X, and I already love R."

[1] "I have started to learn X, and I already love X."

```
[1] "IhavestartedtolearnR,andIalreadyloveR."
```

```
[[1]]
```

```
[1] "I have started to learn R" " and I already love R."
```

```
[[1]]
```

```
[1] "310" "555" "123"
```

```
[[2]]
```

```
[1] "311" "444" "456"
```

24 Data Frames with dplyr

```
'data.frame': 10 obs. of 3 variables:  
 $ num : num 10 20 30 40 50 60 70 80 90 100  
 $ date : Date, format: "2020-01-01" "2020-01-08" ...  
 $ string: chr "word" "word" "word" "word" ...
```

	col1	col2	col3	col4	col5	col6	col7	col8	col9	col10
row1	1	2	3	4	5	6	7	8	9	10
row2	11	12	13	14	15	16	17	18	19	20
row3	21	22	23	24	25	26	27	28	29	30
row4	31	32	33	34	35	36	37	38	39	40
row5	41	42	43	44	45	46	47	48	49	50
row6	51	52	53	54	55	56	57	58	59	60
row7	61	62	63	64	65	66	67	68	69	70
row8	71	72	73	74	75	76	77	78	79	80
row9	81	82	83	84	85	86	87	88	89	90
row10	91	92	93	94	95	96	97	98	99	100

	col1	col2	col3	col4	col5	col6	col7	col8	col9	col10
row1	1	2	3	4	5	6	7	8	9	10
row2	11	12	13	14	15	16	17	18	19	20
row3	21	22	23	24	25	26	27	28	29	30
row4	31	32	33	34	35	36	37	38	39	40
row5	41	42	43	44	45	46	47	48	49	50
row6	51	52	53	54	55	56	57	58	59	60
row7	61	62	63	64	65	66	67	68	69	70
row8	71	72	73	74	75	76	77	78	79	80
row9	81	82	83	84	85	86	87	88	89	90
row10	91	92	93	94	95	96	97	98	99	100

```
[1] 10 10
```

```
[1] 10
```

```
[1] 10
```

```
[1] "data.frame"
```

```
[1] "list"
```

```
      name
1    Max
2   Jane
3   John
4   Tony
5  Janis
6  Helen
```

```
[1] "data.frame"
```

```
[1] 55 35 46 22 60 27
```

```
[1] "numeric"
```

```
[1] 55 35 46 22 60 27
```

```
[1] "numeric"
```

```
      name age
1    Max  55
2   Jane  35
3   John  46
4   Tony  22
5  Janis  60
6  Helen  27
```

	id	name	surname	age	date_start_work	gender	manager_position
1	1	Max	Gordon	55	1985-09-01	M	TRUE
2	2	Jane	Smith	35	2010-10-01	F	FALSE
3	3	John	Don	46	1999-06-01	M	FALSE
4	4	Tony	Price	22	2019-03-01	M	FALSE
5	5	Janis	Jett	60	1980-04-15	F	TRUE
6	6	Helen	Dust	27	2015-02-20	M	FALSE

```
[1] "Jane"
```



```
[1] "Jane"
```

```
gender manager_position
1      M              TRUE
2      F             FALSE
3      M             FALSE
4      M             FALSE
```

```
gender manager_position
1      M              TRUE
2      F             FALSE
3      M             FALSE
4      M             FALSE
```

```
[1] 1 2 3 4 5 6
```

```
[1] 1 2 3 4 5 6
```

```
[1] 1 2 3 4 5 6
```

```
id name surname age date_start_work gender manager_position
5 5 Janis      Jett 60      1980-04-15      F              TRUE
6 6 Helen      Dust 27      2015-02-20      M              FALSE
```

```
id name surname age date_start_work gender manager_position
5 5 Janis      Jett 60      1980-04-15      F              TRUE
6 6 Helen      Dust 27      2015-02-20      M              FALSE
```

```
id          name          surname          age
Min.   :1.0   Length:7      Length:7      Min.   :22.00
1st Qu.:2.5   Class :character  Class :character 1st Qu.:29.50
Median :4.0   Mode  :character  Mode  :character Median :35.00
Mean   :4.0                                     Mean   :39.57
3rd Qu.:5.5                                     3rd Qu.:50.50
Max.   :7.0                                     Max.   :60.00
date_start_work      gender      manager_position      role
Min.   :1980-04-15   Length:7      Mode :logical   Length:7
1st Qu.:1992-07-16   Class :character FALSE:5      Class :character
Median :2010-10-01   Mode  :character TRUE :2      Mode  :character
Mean   :2004-05-06
```

3rd Qu.:2017-02-24

Max. :2020-01-01

eye_color hair_color

Length:7 Length:7

Class :character Class :character

Mode :character Mode :character

	id	name	surname	age	date_start_work	gender	manager_position	role
1	1	Max	Gordon	55	1985-09-01	M	TRUE	director
3	3	John	Don	46	1999-06-01	M	FALSE	analyst
4	4	Tony	Price	22	2019-03-01	M	FALSE	researcher
6	6	Helen	Dust	27	2015-02-20	M	FALSE	analyst
7	7	Mark	Jax	32	2020-01-01	M	FALSE	researcher

	eye_color	hair_color
1	blue	blonde
3	brown	black
4	hazel	brown
6	brown	dark brown
7	brown	brown

	id	name	surname	age	date_start_work	gender	manager_position	role	eye_color
5	5	Janis	Jett	60	1980-04-15	F	TRUE	CEO	blue

	hair_color
5	blonde

	id	name	surname	age	date_start_work	gender	manager_position	role
1	1	Max	Gordon	55	1985-09-01	M	TRUE	director
3	3	John	Don	46	1999-06-01	M	FALSE	analyst
4	4	Tony	Price	22	2019-03-01	M	FALSE	researcher
6	6	Helen	Dust	27	2015-02-20	M	FALSE	analyst
7	7	Mark	Jax	32	2020-01-01	M	FALSE	researcher

	eye_color	hair_color
1	blue	blonde
3	brown	black
4	hazel	brown
6	brown	dark brown
7	brown	brown

	id	name	surname	age	date_start_work	gender	manager_position	role	eye_color
5	5	Janis	Jett	60	1980-04-15	F	TRUE	CEO	blue
									hair_color
5			blonde						

	age	manager_position
277		2

[1] "x is less than y"

[1] "x is less or equal to y"

[1] "x is equal to y"

[1] FALSE

[1] TRUE

[1] TRUE

[1] TRUE

[1] FALSE

[1] TRUE

[1] FALSE FALSE TRUE FALSE

[1] FALSE TRUE FALSE TRUE

[1] TRUE FALSE FALSE FALSE

[1] FALSE TRUE TRUE TRUE

[1] TRUE FALSE TRUE FALSE

[1] TRUE TRUE FALSE TRUE

[1] TRUE TRUE TRUE FALSE

[1] TRUE TRUE FALSE TRUE

[1] FALSE FALSE FALSE TRUE

[1] TRUE FALSE TRUE FALSE

[1] "You win 10\$"

[1] "iteration 1"

[1] "iteration 2"

[1] "iteration 3"

[1] "iteration 4"

[1] "iteration 5"

[1] "iteration 6"

[1] "iteration 7"

[1] "iteration 8"

[1] "iteration 9"

[1] "iteration 10"

[1] "value = 1"

[1] "s = 1"

[1] "-----"

[1] "value = 2"

[1] "s = 3"

[1] "-----"

[1] "value = 3"

[1] "s = 6"

[1] "-----"

[1] "value = 4"

[1] "s = 10"

[1] "-----"

[1] "value = 5"

[1] "s = 15"

[1] "-----"

[1] 5

25 Read sas dataset

26 keeping selected columns

27 dropping columns

To select all columns except certain ones, put a “-” in front of the variable to exclude it. For multiple variables, you can use the function `c()` to combine values into a vector or list. This will select all the variables in surveys except ARM, SITEID

There are many helper functions available with `select()` like: `starts_with()`, `ends_with()`, `contains()` among others. These were imported from the `tidyselect` package. You can put a “-” in front of the helper function to negate it. Here is an example using `contains()`: # Filtering rows To choose rows based on a specific criteria, use `filter()`: To select rows where the planned treatment code equals “Placebo”:

28 Rename

29 Arrange (sorting)

30 convert var names to lower case

```
[1] "studyid" "usubjid" "subjid" "siteid" "sitegr1" "arm"
[7] "trt01p" "trt01pn" "trt01a" "trt01an" "trtsdt" "trtedt"
[13] "trtdur" "avgdd" "cumdose" "age" "agegr1" "agegr1n"
[19] "ageu" "race" "racen" "sex" "ethnic" "saffl"
[25] "ittfl" "efffl" "comp8fl" "comp16fl" "comp24fl" "disconfl"
[31] "dsraefl" "dthfl" "bmibl" "bmiblg1" "heightbl" "weightbl"
[37] "educlvl" "disonsdt" "durdis" "durdsgr1" "visit1dt" "rfstdtc"
[43] "rfendtc" "visnumen" "rfendt" "dcdecod" "dcreascd" "mmsetot"
```

31 Creating new variables

32 bind rows (set operator in SAS)

33 recode (similar to PROC FORMAT in SAS)

34 joins

The dplyr package provides functions to join tables based on shared keys, such as patient IDs. The main types of joins include:

- `inner_join()`
- `left_join()`
- `right_join()`
- `full_join()`
- `semi_join()`
- `anti_join()`

	patient_id	age	gender
1	101	34	F
2	102	45	M
3	103	29	M
4	104	56	F

	patient_id	test	result
1	103	CBC	Normal
2	104	Lipid Panel	High Cholesterol
3	105	Blood Glucose	Elevated
4	106	CBC	Normal

34.1 inner join

`inner_join()`: returns only rows with matching patient IDs in both data frames.

	patient_id	age	gender	test	result
1	103	29	M	CBC	Normal
2	104	56	F	Lipid Panel	High Cholesterol

In this example, only patients with patient_ids 103 and 104 are in both tables, so only their information appears in the result.

34.2 left join

`left_join()` returns all rows from the patients data frame and matches rows from `lab_results` where possible. Unmatched rows in `lab_results` are filled with NA.

	patient_id	age	gender	test	result
1	101	34	F	<NA>	<NA>
2	102	45	M	<NA>	<NA>
3	103	29	M	CBC	Normal
4	104	56	F	Lipid Panel	High Cholesterol

Here, all patients from patients are included, with NA for lab results where no match is found.

34.3 right join

`right_join()` returns all rows from `lab_results`, matching rows from patients where possible. Unmatched rows in patients are filled with NA.

	patient_id	age	gender	test	result
1	103	29	M	CBC	Normal
2	104	56	F	Lipid Panel	High Cholesterol
3	105	NA	<NA>	Blood Glucose	Elevated
4	106	NA	<NA>	CBC	Normal

In this example, all patients with lab results are included, with demographic data from patients where available.

34.4 Full Join

`full_join()` returns all rows from both tables, with NA for missing matches in either table.

	patient_id	age	gender	test	result
1	101	34	F	<NA>	<NA>
2	102	45	M	<NA>	<NA>
3	103	29	M	CBC	Normal
4	104	56	F	Lipid Panel	High Cholesterol
5	105	NA	<NA>	Blood Glucose	Elevated
6	106	NA	<NA>	CBC	Normal

34.5 Semi Join

`semi_join()` returns only the rows in `patients` that have a match in `lab_results`, without bringing in columns from `lab_results`.

	patient_id	age	gender
1	103	29	M
2	104	56	F

Only patients with lab results are included here (patients 103 and 104).

34.6 Anti join

`anti_join()` returns only the rows in `patients` that do not have a match in `lab_results`.

	patient_id	age	gender
1	101	34	F
2	102	45	M

35 Reshaping data with tidyr

36 counting with `n()`

When working with data, we often want to know the number of observations found for each factor or combination of factors. For this task, `dplyr` provides `count()`. For example, if we wanted to count the number of rows of data for each sex, we would do:

37 dplyr distinct()

The `distinct()` function in `dplyr` is used to return unique/distinct rows from a data frame or tibble. It removes duplicate rows based on the values in specified columns or all columns if none are specified.

```
# A tibble: 3 x 2
  TRT01P          TRT01PN
  <chr>          <dbl>
1 Placebo          0
2 Xanomeline High Dose 81
3 Xanomeline Low Dose  54
```

38 `case_when()`

The `case_when()` function in `dplyr` is a powerful tool for creating new variables based on multiple conditions. It allows you to specify a series of conditions and corresponding values to assign when those conditions are met.

39 create seq no.

40 Lists: lapply, purrr

```
$a  
[1] 2
```

```
$b  
[1] 11
```

41 Exercises

1. Using `across()`, standardize $(x - \text{mean}) / \text{sd}$ numeric columns.
2. Create a row-wise mean of `age` and `wt`.
3. Split `df` by `grp` and compute group means with `lapply` or `purrr::map`.

42 String Functions

43 String Functions in Base R

Strings (or character vectors) are a fundamental data type in R, and base R provides a variety of functions to manipulate and analyze strings. Here are some commonly used string functions in base R:

43.1 Basic String Functions

```
[1] 13 10 12
```

```
[1] "hello, world!" "r is great"    "data science"
```

```
[1] "HELLO, WORLD!" "R IS GREAT"    "DATA SCIENCE"
```

```
[1] "Hello" "R is " "Data "
```

```
[1] "Hello R"
```

```
[1] "HelloR"
```

```
[1] "Hello, World!"
```

43.2 Pattern Matching and Replacement

```
[1] 2
```

```
[1] FALSE TRUE FALSE
```

```
[1] "Hello, World!" "R is awesome" "Data Science"
```

```
[1] "Hello,_World!" "R_is_great"    "Data_Science"
```

43.3 String Splitting and Joining

```
[[1]]  
[1] "Hello," "World!"  
  
[[2]]  
[1] "R"      "is"      "great"  
  
[[3]]  
[1] "Data"    "Science"  
  
[1] "Hello," "World!" "R"      "is"      "great" "Data"    "Science"
```

43.4 Regular Expressions

```
[1] -1  1 -1  
attr(,"match.length")  
[1] -1  1 -1  
attr(,"index.type")  
[1] "chars"  
attr(,"useBytes")  
[1] TRUE
```

```
[[1]]  
[1] 7  
attr(,"match.length")  
[1] 1  
attr(,"index.type")  
[1] "chars"  
attr(,"useBytes")  
[1] TRUE
```

```
[[2]]  
[1] 2 5  
attr(,"match.length")  
[1] 1 1  
attr(,"index.type")  
[1] "chars"  
attr(,"useBytes")  
[1] TRUE
```

```
[[3]]
[1] 5
attr(,"match.length")
[1] 1
attr(,"index.type")
[1] "chars"
attr(,"useBytes")
[1] TRUE
```

```
[[1]]
[1] " "
```

```
[[2]]
[1] " " " " "
```

```
[[3]]
[1] " "
```

43.5 Example Usage

```
[1] "hello,_world!" "r_is_great"    "data_science"
```

These functions provide a solid foundation for string manipulation in R. For more advanced string operations, you # using the **stringr** package, which offers a more consistent and user-friendly interface for string handling.

44 Overall differences

We'll begin with a lookup table between the most important stringr functions and their base R equivalents.

Overall the main differences between base R and stringr are:

1. stringr functions start with `str_` prefix; base R string functions have no consistent naming scheme.
2. The order of inputs is usually different between base R and stringr. In base R, the `pattern` to match usually comes first; in stringr, the `string` to manipulate always comes first. This makes stringr easier to use in pipes, and with `lapply()` or `purrr::map()`.
3. Functions in stringr tend to do less, where many of the string processing functions in base R have multiple purposes.
4. The output and input of stringr functions has been carefully designed. For example, the output of `str_locate()` can be fed directly into `str_sub()`; the same is not true of `regexpr()` and `substr()`.
5. Base functions use arguments (like `perl`, `fixed`, and `ignore.case`) to control how the pattern is interpreted. To avoid dependence between arguments, stringr instead uses helper functions (like `fixed()`, `regex()`, and `coll()`).

Next we'll walk through each of the functions, noting the similarities and important differences. These examples are adapted from the stringr documentation and here they are contrasted with the analogous base R operations.

stringr	base R
<code>str_detect(string, pattern)</code>	<code>grepl(pattern, x)</code>
<code>str_dup(string, times)</code>	<code>strrep(x, times)</code>
<code>str_extract(string, pattern)</code>	<code>regmatches(x, m = regexpr(pattern, text))</code>
<code>str_extract_all(string, pattern)</code>	<code>regmatches(x, m = gregexpr(pattern, text))</code>
<code>str_length(string)</code>	<code>nchar(x)</code>
<code>str_locate(string, pattern)</code>	<code>regexpr(pattern, text)</code>
<code>str_locate_all(string, pattern)</code>	<code>gregexpr(pattern, text)</code>
<code>str_match(string, pattern)</code>	<code>regmatches(x, m = regexec(pattern, text))</code>
<code>str_order(string)</code>	<code>order(...)</code>
<code>str_replace(string, pattern, replacement)</code>	<code>sub(pattern, replacement, x)</code>
<code>str_replace_all(string, pattern, replacement)</code>	<code>gsub(pattern, replacement, x)</code>
<code>str_sort(string)</code>	<code>sort(x)</code>
<code>str_split(string, pattern)</code>	<code>strsplit(x, split)</code>
<code>str_sub(string, start, end)</code>	<code>substr(x, start, stop)</code>
<code>str_subset(string, pattern)</code>	<code>grep(pattern, x, value = TRUE)</code>
<code>str_to_lower(string)</code>	<code>tolower(x)</code>
<code>str_to_title(string)</code>	<code>tools::toTitleCase(text)</code>
<code>str_to_upper(string)</code>	<code>toupper(x)</code>
<code>str_trim(string)</code>	<code>trimws(x)</code>
<code>str_which(string, pattern)</code>	<code>grep(pattern, x)</code>
<code>str_wrap(string)</code>	<code>strwrap(x)</code>

45 Detect matches

45.1 `str_detect()`: Detect the presence or absence of a pattern in a string

Suppose you want to know whether each word in a vector of fruit names contains an “a”.

```
[1] TRUE TRUE TRUE TRUE
```

```
[1] TRUE TRUE TRUE TRUE
```

In base you would use `grep1()` (see the “l” and think logical) while in `stringr` you use `str_detect()` (see the verb “detect” and think of a yes/no action).

45.2 `str_which()`: Find positions matching a pattern

Now you want to identify the positions of the words in a vector of fruit names that contain an “a”.

```
[1] 1 2 3 4
```

```
[1] 1 2 3 4
```

In base you would use `grep()` while in `stringr` you use `str_which()` (by analogy to `which()`).

45.3 str_count(): Count the number of matches in a string

How many “a”s are in each fruit?

```
[1] 1 3 1 1
```

```
[1] 1 3 1 1
```

This information can be gleaned from `gregexpr()` in `base`, but you need to look at the `match.length` attribute as the vector uses a length-1 integer vector (-1) to indicate no match.

45.4 str_locate(): Locate the position of patterns in a string

Within each fruit, where does the first “p” occur? Where are all of the “p”s?

List of 3

```
$ : int [1:2] 1 3
..- attr(*, "match.length")= int [1:2] 1 1
..- attr(*, "index.type")= chr "chars"
..- attr(*, "useBytes")= logi TRUE
$ : int -1
..- attr(*, "match.length")= int -1
..- attr(*, "index.type")= chr "chars"
..- attr(*, "useBytes")= logi TRUE
$ : int [1:2] 2 3
..- attr(*, "match.length")= int [1:2] 1 1
..- attr(*, "index.type")= chr "chars"
..- attr(*, "useBytes")= logi TRUE
```

```
      start end
[1,]      1   1
[2,]     NA  NA
[3,]      2   2
```

```
[[1]]
      start end
[1,]      1   1
[2,]      3   3
```

```
[[2]]
      start end
```

```
[[3]]
      start end
[1,]      2   2
[2,]      3   3
```

45.5 `str_subset()`: Keep strings matching a pattern, or find positions

We may want to retrieve strings that contain a pattern of interest:

```
character(0)
```

```
character(0)
```

45.6 `str_extract()`: Extract matching patterns from a string

We may want to pick out certain patterns from a string, for example, the digits in a shopping list:

```
[1] "4" "10" "2"
```

```
[[1]]
[1] "apples" "x"
```

```
[[2]]
[1] "bag" "of" "flour"
```

```
[[3]]
character(0)
```

```
[[4]]
[1] "milk" "x"
```

```
[1] "4" NA "10" "2"
```



```

[[1]]
[1] "apples" "x"

[[2]]
[1] "bag"      "of"      "flour"

[[3]]
character(0)

[[4]]
[1] "milk" "x"

```

Base R requires the combination of `regexpr()` with `regmatches()`; but note that the strings without matches are dropped from the output. `stringr` provides `str_extract()` and `str_extract_all()`, and the output is always the same length as the input.

45.7 `str_match()`: Extract matched groups from a string

We may also want to extract groups from a string. Here I'm going to use the scenario from Section 14.4.3 in [R for Data Science](#).

```

[1] "The birch canoe slid on the smooth planks."
[2] "Glue the sheet to the dark blue background."
[3] "It's easy to tell the depth of a well."
[4] "These days a chicken leg is a rare dish."
[5] "Rice is often served in round bowls."
[6] "The juice of lemons makes fine punch."

```

```

      [,1]      [,2] [,3]
[1,] "The birch" "The" "birch"
[2,] "the sheet" "the" "sheet"
[3,] "the depth" "the" "depth"
[4,] "The juice" "The" "juice"

```

```

      [,1]      [,2] [,3]
[1,] "The birch" "The" "birch"
[2,] "the sheet" "the" "sheet"
[3,] "the depth" "the" "depth"
[4,] NA         NA   NA
[5,] NA         NA   NA
[6,] "The juice" "The" "juice"

```

As for extracting the full match base R requires the combination of two functions, and inputs with no matches are dropped from the output.

46 Manage lengths

46.1 `str_length()`: The length of a string

To determine the length of a string, base R uses `nchar()` (not to be confused with `length()` which gives the length of vectors, etc.) while `stringr` uses `str_length()`.

```
[1] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
```

```
[1] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
```

There are some subtle differences between base and `stringr` here. `nchar()` requires a character vector, so it will return an error if used on a factor. `str_length()` can handle a factor input.

```
Error in nchar(factor("abc")): 'nchar()' requires a character vector
```

```
[1] 3
```

Note that “characters” is a poorly defined concept, and technically both `nchar()` and `str_length()` returns the number of code points. This is usually the same as what you’d consider to be a character, but not always:

```
[1] "ü" "ü"
```

```
[1] 1 2
```

```
[1] 1 2
```

46.2 `str_pad()`: Pad a string

To pad a string to a certain width, use `stringr`'s `str_pad()`. In base R you could use `sprintf()`, but unlike `str_pad()`, `sprintf()` has many other functionalities.

```
[1] "                               Sriram"

[1] "Sriram                               "

      [,1]
[1,] "                               Sriram"
[2,] "Sriram                               "
[3,] "                               Sriram"
```

46.3 `str_trunc()`: Truncate a character string

The `stringr` package provides an easy way to truncate a character string: `str_trunc()`. Base R has no function to do this directly.

```
      [,1]
[1,] "This string is mo..."
[2,] "...s moderately long"
[3,] "This stri...ely long"
```

46.4 `str_trim()`: Trim whitespace from a string

Similarly, `stringr` provides `str_trim()` to trim whitespace from a string. This is analogous to base R's `trimws()` added in R 3.3.0.

```
[1] "String with trailing and leading white space"

[1] "String with trailing and leading white space"

[1] "String with trailing and leading white space"

[1] "String with trailing and leading white space"
```

The stringr function `str_squish()` allows for extra whitespace within a string to be trimmed (in contrast to `str_trim()` which removes whitespace at the beginning and/or end of string). In base R, one might take advantage of `gsub()` to accomplish the same effect.

```
[1] "String with trailing, middle, and leading white space"
```

```
[1] "String with excess, trailing and leading white space"
```

46.5 `str_wrap()`: Wrap strings into nicely formatted paragraphs

`strwrap()` and `str_wrap()` use different algorithms. `str_wrap()` uses the famous [Knuth-Plass algorithm](#).

```
Four score and seven years ago our fathers brought forth on
this continent, a new nation, conceived in Liberty, and
dedicated to the proposition that all men are created
equal.
```

```
Four score and seven years ago our fathers brought forth
on this continent, a new nation, conceived in Liberty, and
dedicated to the proposition that all men are created equal.
```

Note that `strwrap()` returns a character vector with one element for each line; `str_wrap()` returns a single string containing line breaks.

47 Mutate strings

47.1 `str_replace()`: Replace matched patterns in a string

To replace certain patterns within a string, `stringr` provides the functions `str_replace()` and `str_replace_all()`. The base R equivalents are `sub()` and `gsub()`. Note the difference in default input order again.

```
[1] "-pple"      "b-nana"      "p-ar"        "p-neapple"
[1] "-ppl-"      "b-n-n-"      "p--r"        "p-n--ppl-"
[1] "-pple"      "b-nana"      "p-ar"        "p-neapple"
[1] "-ppl-"      "b-n-n-"      "p--r"        "p-n--ppl-"
```

47.2 `case`: Convert case of a string

Both `stringr` and base R have functions to convert to upper and lower case. Title case is also provided in `stringr`.

```
[1] "THE QUICK BROWN DOG"
[1] "the quick brown dog"
[1] "The Quick Brown Dog"
[1] "THE QUICK BROWN DOG"
[1] "the quick brown dog"
[1] "The Quick Brown Dog"
```

In `stringr` we can control the locale, while in base R locale distinctions are controlled with global variables. Therefore, the output of your base R code may vary across different computers with different global settings.

```
[1] "I"
```

```
[1] "İ"
```

48 Join and split

48.1 `str_flatten()`: Flatten a string

If we want to take elements of a string vector and collapse them to a single string we can use the `collapse` argument in `paste()` or use `stringr`'s `str_flatten()`.

```
[1] "a-b-c-d-e-f-g-h-i-j-k-l-m-n-o-p-q-r-s-t-u-v-w-x-y-z"
```

```
[1] "a-b-c-d-e-f-g-h-i-j-k-l-m-n-o-p-q-r-s-t-u-v-w-x-y-z"
```

The advantage of `str_flatten()` is that it always returns a vector the same length as its input; to predict the return length of `paste()` you must carefully read all arguments.

48.2 `str_dup()`: duplicate strings within a character vector

To duplicate strings within a character vector use `strrep()` (in R 3.3.0 or greater) or `str_dup()`:

```
[1] "appleapple"    "pearpear"      "bananabanana"
```

```
[1] "apple"          "pearpear"      "bananabanabanana"
```

```
[1] "appleapple"    "pearpear"      "bananabanana"
```

```
[1] "apple"          "pearpear"      "bananabanabanana"
```


48.3 `str_split()`: Split up a string into pieces

To split a string into pieces with breaks based on a particular pattern match stringr uses `str_split()` and base R uses `strsplit()`. Unlike most other functions, `strsplit()` starts with the character vector to modify.

```
[[1]]
[1] "apples" "oranges" "pears"  "bananas"
```

```
[[2]]
[1] "pineapples" "mangos"      "guavas"
```

```
[[1]]
[1] "apples" "oranges" "pears"  "bananas"
```

```
[[2]]
[1] "pineapples" "mangos"      "guavas"
```

The stringr package's `str_split()` allows for more control over the split, including restricting the number of possible matches.

```
[[1]]
[1] "apples"          "oranges"          "pears and bananas"
```

```
[[2]]
[1] "pineapples" "mangos"      "guavas"
```

```
[[1]]
[1] "apples"          "oranges and pears and bananas"
```

```
[[2]]
[1] "pineapples"      "mangos and guavas"
```

48.4 `str_glue()`: Interpolate strings

It's often useful to interpolate varying values into a fixed string. In base R, you can use `sprintf()` for this purpose; stringr provides a wrapper for the more general purpose [glue](#) package.

[1] "My name is Fred my age next year is 51 and my anniversary is Saturday, October 12, 1991

My name is Fred, my age next year is 51, and my anniversary is Saturday, October 12, 1991.

49 Order strings

49.1 `str_order()`: Order or sort a character vector

Both base R and `stringr` have separate functions to order and sort strings.

```
[1] 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25
[26] 26
```

```
[1] "a" "b" "c" "d" "e" "f" "g" "h" "i" "j" "k" "l" "m" "n" "o" "p" "q" "r" "s"
[20] "t" "u" "v" "w" "x" "y" "z"
```

```
[1] 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25
[26] 26
```

```
[1] "a" "b" "c" "d" "e" "f" "g" "h" "i" "j" "k" "l" "m" "n" "o" "p" "q" "r" "s"
[20] "t" "u" "v" "w" "x" "y" "z"
```

Some options in `str_order()` and `str_sort()` don't have analogous base R options. For example, the `stringr` functions have a `locale` argument to control how to order or sort. In base R the locale is a global setting, so the outputs of `sort()` and `order()` may differ across different computers. For example, in the Norwegian alphabet, å comes after z:

```
[1] "a" "å" "z"
```

```
[1] "a" "z" "å"
```

The `stringr` functions also have a `numeric` argument to sort digits numerically instead of treating them as strings.

```
[1] "100a10" "100a5" "2a" "2b"
```

```
[1] "2a" "2b" "100a5" "100a10"
```

Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

`filter`, `lag`

The following objects are masked from 'package:base':

`intersect`, `setdiff`, `setequal`, `union`

50 date and time functions

50.0.1 Date and Time Base Functions in R

Date and time manipulation is a common task in data analysis. R provides several base functions to work with dates and times. Here are some of the most commonly used base R functions for handling date and time data:

```
[1] "2025-11-20"
```

```
[1] "2025-11-20 10:24:48 UTC"
```

50.0.2 Creating Date and Time Objects

50.0.3 Year Formats

Code	Meaning	Example (<code>format(dt, ...)</code>)
<code>%Y</code>	4-digit year	"2025"
<code>%y</code>	2-digit year (00–99)	"25"
<code>%C</code>	Century (year / 100, 00–99)	"20" (for 2025; OS-dep.)

50.0.4 Month Formats

Code	Meaning	Example
<code>%m</code>	Month number (01–12)	"11"
<code>%b</code>	Abbrev. month name (locale)	"Nov"
<code>%B</code>	Full month name (locale)	"November"
<code>%h</code>	Same as <code>%b</code> (abbrev. month name)	"Nov"

50.0.5 Day Formats

Code	Meaning	Example
%d	Day of month (01–31)	"14"
%e	Day of month (1–31, padded with space)	"14" (or " 7" for 7th; OS-dep.)
%j	Day of year (001–366)	"318"

50.0.6 Weekday Formats

Code	Meaning	Example
%a	Abbrev. weekday name (locale)	"Fri"
%A	Full weekday name (locale)	"Friday"
%w	Weekday number (0–6, 0 = Sunday)	"5"
%u	ISO weekday number (1–7, 1 = Monday) (OS-dep.)	"5" (Friday)

50.0.7 Time Formats

Code	Meaning	Example
%H	Hour (00–23)	"14"
%I	Hour (01–12)	"02"
%M	Minute (00–59)	"30"
%S	Second (00–61, allows for leap)	"05"
%p	AM/PM indicator (locale)	"PM"

###Locale-dependent “full date/time” codes

###These depend on your system locale (language + region):

Code	Meaning	Example
%c	Date and time representation	"Fri Nov 14 16:05:30 2025"
%x	Date representation	"11/14/25" (varies by locale)
%X	Time representation	"16:05:30"

[1] "2025-11-14"

[1] "11/14/25"

[1] "November 14, 2025"

```
[1] "Fri Nov 14 16:05:30 2025"
```

```
[1] "11/14/25"
```

```
[1] "16:05:30"
```

SAS informat	R parsing
ddmmyy10.	as.Date(x, "%d/%m/%Y")
ddmmyyd10.	as.Date(x, "%d-%m-%Y")
ddmmyyp10.	as.Date(x, "%d.%m.%Y")
mmddy10.	as.Date(x, "%m/%d/%Y")
date9.	as.Date(x, "%d%b%Y")
julian7.	as.Date(strptime(x, "%Y%j"))
time8.	as.POSIXct(x, "%H:%M:%S")
time10.	as.POSIXct(x, "%I:%M:%S%p") (if am/pm)
datetime18.	as.POSIXct(x, "%d%b%Y:%H:%M:%S")
datetime20.	as.POSIXct(x, "%d%b%Y:%I:%M:%S%p")

SAS format	Meaning	R equivalent
worddate18.	Month date, year (e.g., February 23, 2003)	format(d, "%B %d, %Y")
weekdate24.	Weekday, month date, year	format(d, "%A, %B %d, %Y")
year.	numeric year	as.integer(format(d, "%Y"))
month.	month number (1–12)	as.integer(format(d, "%m"))
day.	day of month	as.integer(format(d, "%d"))
weekday.	day of week (1–7)	as.integer(format(d, "%u")) (Mon = 1)

```
[1] "2023-10-01"
```

```
[1] "2023-10-01 12:34:56 UTC"
```

```
[1] "Year: 2023 Month: 10 Day: 01"
```

```
[1] "Hour: 12 Minute: 34 Second: 56"
```

```
[1] "Tomorrow: 2025-11-21 Yesterday: 2025-11-19"
```

```
[1] "Difference in days: 781"
```

```
[1] "November 20, 2025"
```

```
[1] "2025-11-20 10:24:48"
```

These functions allow you to create, manipulate, and format date and time data in R. For more advanced date and time handling, you might consider using the **lubridate** package, which provides a more user-friendly interface for working with dates and times.

A practical, CDISC-friendly guide to working with **dates/times** and implementing **partial date imputation** in R using **base R** and **lubridate**.

50.1 1. Date & Time Basics in R

50.1.1 1.1 Core classes

- **Date**: calendar dates without time.
- **POSIXct**: seconds since 1970-01-01 (compact, good for storage/arithmetic).
- **POSIXlt**: list of components (year, mon, mday, hour, min, sec) — convenient to extract parts.

50.1.2 1.2 Converting numerics to Date (origins matter)

Different systems use different **origins** for numeric dates: - Excel: 1900-01-01 (beware 1900 leap-year bug in older Excels) - SAS: 1960-01-01 - R: 1970-01-01

```
[1] "1960-01-01" "1960-06-08" "1961-04-09" "2002-11-30"
```

```
[1] "1970-01-01" "1970-06-09" "1971-04-10" "2012-11-30"
```

```
[1] "1900-01-01" "1900-06-09" "1901-04-10" "1942-12-01"
```

50.1.3 1.3 Custom date formats


```
x1 <- c("01/05/1965", "08/16/1975")
as.Date(x1, "%m/%d/%Y")

x2 <- c("05-01-1965", "16-08-1975")
as.Date(x2, "%d-%m-%Y")

x3 <- c("01MAY1965", "16AUG1975")
as.Date(x3, "%d%b%Y")

# Unknown format? Try multiple patterns
x4 <- c("01MAY1965MORNING", "16AUGUST1975")
as.Date(x4, tryFormats = c("%d%b%Y", "%d%B%Y"))
```

50.1.4 1.4 Datetime strings (ISO 8601 & time zones)

```
x <- c("2015-10-19T10:15", "2018-12-19T23:05")
# Keep the literal 'T' in the format string
as.POSIXct(x, format = "%Y-%m-%dT%H:%M", tz = "UTC")

# Extract parts
xt <- as.POSIXlt(x, format = "%Y-%m-%dT%H:%M")
format(xt, "%Y-%m-%d") # date part
format(xt, "%H:%M")    # time part
```

50.2 2. Lubridate: Fast, Friendly Parsing & Arithmetic

Install & load: `install.packages("lubridate"); library(lubridate)`

50.2.1 2.1 Intuitive parsers

- `ymd()`, `mdy()`, `dmy()` for dates
- `ymd_hms()`, `ymd_hm()`, `ymd_h()` for date-times

```
ymd("20210529")
dmy("29/05/2021")
ymd_hm("2019/01/19T17:15") # delimiter agnostic
```

50.2.2 2.2 Accessors and rounding

```
x <- ymd_hms("2009-08-03 12:01:59")
year(x); month(x); mday(x); wday(x); hour(x); minute(x); second(x)

floor_date(x, "month") # 2009-08-01
ceiling_date(x, "month") # 2009-09-01
round_date(x, "month") # 2009-08-01
rollback(x) # last day of previous month
```

50.2.3 2.3 Timespans

- **Durations:** exact seconds, e.g., `ddays(2)`
- **Periods:** human units, e.g., `months(1)`
- **Intervals:** start-end pair, e.g., `interval(start, end)`

```
start <- ymd("2015-10-19")
end <- ymd("2018-12-19")
int <- interval(start, end)
as.duration(int) # exact seconds
as.period(int) # years/months/days
```

50.2.4 3 ADaM-Style Partial Date Imputation (Start & End)

Imputation rules vary by study; always follow the SAP. A common pattern:

50.2.5 Start date (AESTDTC)

- YYYY-MM-DD → use as is
- YYYY-MM → impute **day** = **01** (first of month)
- YYYY → impute **month** = **01**, **day** = **01** (01-Jan)

50.2.6 End date (AEENDTC)

- YYYY-MM-DD → use as is
- YYYY-MM → impute **last day of that month**
- YYYY → impute **31-Dec**

50.2.7 Censoring by death / last alive date

Additionally, bound AEENDT by death / last alive date (when present):

- CENSOR = coalesce(DTHDT, LASTALVDT)
- AEENDT = min(imputed_end, CENSOR)

50.2.8 Imputation flags

Add flags:

- AESTDTF, AEENDTF:
 - "D" → day imputed
 - "M,D" → month & day imputed
- Add "C" if we had to **censor the end date** at DTHDT / LASTALVDT.

50.3 4. Durations & Analysis Variables

50.3.1 4.1 AE duration (AEDUR) in days

51 Using difftime (base) or as.numeric on Date differences

```
AEDUR <- as.integer(out$AEENDT - out$ASTDT)
```

51.0.1 6.2 Handy lubridate parsers (subset)

- Dates: `ymd()`, `mdy()`, `dmy()`, `yq()`
 - Datetimes: `ymd_hms()`, `ymd_hm()`, `ymd_h()`
 - Times: `hms()`, `hm()`, `ms()`
 - Accessors: `year()`, `month()`, `mday()`, `wday()`, `hour()`, `minute()`, `second()`, `date()`
 - Rounding: `floor_date()`, `ceiling_date()`, `round_date()`, `rollback()`
-

52 Reading SAS Datasets (+ Cleaning)

Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

filter, lag

The following objects are masked from 'package:base':

intersect, setdiff, setequal, union

We try to read a SAS dataset (e.g., SDTM DM). If not present, we **synthesize** an example.

```
tibble [306 x 25] (S3: tbl_df/tbl/data.frame)
 $ STUDYID : chr [1:306] "CDISCPILLOT01" "CDISCPILLOT01" "CDISCPILLOT01" "CDISCPILLOT01" ...
 ..- attr(*, "label")= chr "Study Identifier"
 $ DOMAIN  : chr [1:306] "DM" "DM" "DM" "DM" ...
 ..- attr(*, "label")= chr "Domain Abbreviation"
 $ USUBJID : chr [1:306] "01-701-1015" "01-701-1023" "01-701-1028" "01-701-
1033" ...
 ..- attr(*, "label")= chr "Unique Subject Identifier"
 $ SUBJID   : chr [1:306] "1015" "1023" "1028" "1033" ...
 ..- attr(*, "label")= chr "Subject Identifier for the Study"
 $ RFSTDTC  : chr [1:306] "2014-01-02" "2012-08-05" "2013-07-19" "2014-03-
18" ...
 ..- attr(*, "label")= chr "Subject Reference Start Date/Time"
 $ RFENDTC  : chr [1:306] "2014-07-02" "2012-09-02" "2014-01-14" "2014-04-
14" ...
 ..- attr(*, "label")= chr "Subject Reference End Date/Time"
 $ RFXSTDTC : chr [1:306] "2014-01-02" "2012-08-05" "2013-07-19" "2014-03-
18" ...
 ..- attr(*, "label")= chr "Date/Time of First Study Treatment"
 $ RFXENDTC : chr [1:306] "2014-07-02" "2012-09-01" "2014-01-14" "2014-03-
31" ...
```

```

    ..- attr(*, "label")= chr "Date/Time of Last Study Treatment"
$ RFICDTC : chr [1:306] "" "" "" "" ...
    ..- attr(*, "label")= chr "Date/Time of Informed Consent"
$ RFPENDTC: chr [1:306] "2014-07-02T11:45" "2013-02-18" "2014-01-14T11:10" "2014-
09-15" ...
    ..- attr(*, "label")= chr "Date/Time of End of Participation"
$ DTHDTC  : chr [1:306] "" "" "" "" ...
    ..- attr(*, "label")= chr "Date/Time of Death"
$ DTHFL   : chr [1:306] "" "" "" "" ...
    ..- attr(*, "label")= chr "Subject Death Flag"
$ SITEID  : chr [1:306] "701" "701" "701" "701" ...
    ..- attr(*, "label")= chr "Study Site Identifier"
$ AGE     : num [1:306] 63 64 71 74 77 85 59 68 81 84 ...
    ..- attr(*, "label")= chr "Age"
$ AGEU    : chr [1:306] "YEARS" "YEARS" "YEARS" "YEARS" ...
    ..- attr(*, "label")= chr "Age Units"
$ SEX     : chr [1:306] "F" "M" "M" "M" ...
    ..- attr(*, "label")= chr "Sex"
$ RACE    : chr [1:306] "WHITE" "WHITE" "WHITE" "WHITE" ...
    ..- attr(*, "label")= chr "Race"
$ ETHNIC  : chr [1:306] "HISPANIC OR LATINO" "HISPANIC OR LATINO" "NOT HISPANIC OR LATINO" ...
    ..- attr(*, "label")= chr "Ethnicity"
$ ARMCD   : chr [1:306] "Pbo" "Pbo" "Xan_Hi" "Xan_Lo" ...
    ..- attr(*, "label")= chr "Planned Arm Code"
$ ARM     : chr [1:306] "Placebo" "Placebo" "Xanomeline High Dose" "Xanomeline Low Dose" ..
    ..- attr(*, "label")= chr "Description of Planned Arm"
$ ACTARMCD: chr [1:306] "Pbo" "Pbo" "Xan_Hi" "Xan_Lo" ...
    ..- attr(*, "label")= chr "Actual Arm Code"
$ ACTARM  : chr [1:306] "Placebo" "Placebo" "Xanomeline High Dose" "Xanomeline Low Dose" ..
    ..- attr(*, "label")= chr "Description of Actual Arm"
$ COUNTRY : chr [1:306] "USA" "USA" "USA" "USA" ...
    ..- attr(*, "label")= chr "Country"
$ DMDTC   : chr [1:306] "2013-12-26" "2012-07-22" "2013-07-11" "2014-03-
10" ...
    ..- attr(*, "label")= chr "Date/Time of Collection"
$ DMDY    : num [1:306] -7 -14 -8 -8 -7 -21 NA -9 -13 -7 ...
    ..- attr(*, "label")= chr "Study Day of Collection"

```

52.1 Handling Labels & Missing

52.2 Labelled to Factor (if needed)

52.3 Common Cleaning

53 Exercises

1. Read another SAS dataset (e.g., `sv.sas7bdat`) if available. If not, create a synthetic tibble.
2. Write a function to trim character whitespace for all character columns.
3. Make a clean factor for `ARM` with levels `Placebo < Active`.

54 Base R Functions & Apply Family

55 Common Utilities

```
[1] 55
```

```
[1] 5.5
```

```
[1] 3.02765
```

```
[1] 9.166667
```

0%	25%	50%	75%	100%
1.00	3.25	5.50	7.75	10.00

```
[1] 0.0 0.1 0.2 0.3 0.4 0.5 0.6 0.7 0.8 0.9 1.0
```

```
[1] 5 5 5
```

56 Apply Family

```
[1] 4 5 6
```

```
[1] 2 5 8
```

```
a  b  
2 11
```

```
[1] 11 13 15
```

57 Subsetting Essentials

```
[1] 10
```

```
      id val  
2     2  20  
3     3  30
```

58 Exercises

1. Use `apply` to get the max per column of a numeric matrix.
2. Write a base R snippet to compute IQR for each column of `mtcars`.
3. Compare `lapply` vs `sapply` in behavior on a list with mixed types.

59 Custom Functions & Validation

60 Writing Functions

[1] 1.5

61 Error Handling

[1] Inf

62 Unit Testing with testthat

Install once: `install.packages(c("testthat","devtools","usethis","roxygen2"))`

Create `tests/testthat/test-safe_mean.R`:

Test passed with 2 successes.

63 Document with roxygen2

Run:

64 Exercises

1. Write `winsorize(x, probs=c(0.05,0.95))` and test it.
2. Create `validate_columns(df, required=c("USUBJID","AGE"))` and add tests.
3. Add roxygen docs and build help pages.

65 R Package Development

65.1 Setup

65.2 Create a Package

65.3 Add a Function

Create `R/safe_mean.R` and its tests (see previous chapter).

65.4 Build, Install, Check

65.5 Vignette & Website

Exercise: Package-ize a small utility set (`convert_blanks_to_na`, `validate_columns`, etc.) with docs and tests.

66 Git in RStudio (Setup & Auth)

66.1 One-Time Setup

- Install Git and ensure `git --version` works.
- In R:

66.2 Initialize Git for the Current Project

66.3 Connect to GitHub

- Create a GitHub account.
- In R:

Or set up SSH keys via RStudio (Tools > Global Options > Git/SVN).

66.4 Typical Workflow

1. Stage changes (Git pane in RStudio).
2. Commit with a clear message.
3. Push to origin (GitHub).

66.5 Remove Git from a Project (macOS/RStudio)

- In Finder/Terminal, delete the hidden `.git` folder in the project root (careful!).
- Or from Terminal at project root:

```
rm -rf .git
```

- Reopen project in RStudio; Git pane will disappear.

Exercises - Create a new repo for this Quarto course and push it. - Branch, make a change, open a Pull Request on GitHub.

67 Creating ADaM: ADSL from SDTM-like Inputs

Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

filter, lag

The following objects are masked from 'package:base':

intersect, setdiff, setequal, union

Attaching package: 'lubridate'

The following objects are masked from 'package:base':

date, intersect, setdiff, union

We simulate **minimal** SDTM-like DM and EX to illustrate ADSL creation. If available, replace with your own `data/sdtm/*.sas7bdat`.

67.1 Build ADSL

A tibble: 10 x 10

	STUDYID	USUBJID	TRT01P	TRT01PN	AGE	SEX	RANDDT	TRTSDT	TRT01A
	<chr>	<chr>	<chr>	<int>	<dbl>	<chr>	<date>	<date>	<chr>
1	XYZ123	XYZ-001	Placebo	1	55	M	2025-01-18	2025-01-19	Placebo
2	XYZ123	XYZ-002	Active	2	62	F	2025-01-24	2025-01-26	Active
3	XYZ123	XYZ-003	Placebo	1	47	M	2025-02-01	2025-02-01	Placebo
4	XYZ123	XYZ-004	Active	2	50	F	2025-02-03	2025-02-05	Active

```

5 XYZ123 XYZ-005 Placebo      1    71 M    2025-01-29 2025-02-01 Placebo
6 XYZ123 XYZ-006 Active      2    66 F    2025-01-27 2025-01-30 Active
7 XYZ123 XYZ-007 Placebo      1    45 M    2025-01-23 2025-01-25 Placebo
8 XYZ123 XYZ-008 Active      2    59 F    2025-01-29 2025-01-30 Active
9 XYZ123 XYZ-009 Placebo      1    53 M    2025-01-23 2025-01-26 Placebo
10 XYZ123 XYZ-010 Active      2    68 F    2025-01-21 2025-01-21 Active
# i 1 more variable: TRT01AN <int>

```

Note: Real ADSL creation must follow **ADaM IG** (derive flags, dates, imputations, populations). This example is educational only.

Exercises 1. Add analysis populations (e.g., SAFFL, FASFL) based on simple rules. 2. Derive AGEGR1 as <65 / 65 and use ordered factor. 3. Add a treatment end date TRTEDT and compute treatment duration.

68 TLFs: Table, Figure, Listing

We reuse `ads1` from the previous chapter (or synthesize if missing).

68.1 Table 1: Baseline Characteristics by Treatment

	Placebo	Xanomeline High Dose	Xanomeline Low Dose	Overall
	(N=86)	(N=84)	(N=84)	(N=254)
SEX				
Female	53 (61.6%)	40 (47.6%)	50 (59.5%)	143 (56.3%)
Male	33 (38.4%)	44 (52.4%)	34 (40.5%)	111 (43.7%)
Age				
Mean (SD)	75.2 (8.59)	74.4 (7.89)	75.7 (8.29)	75.1 (8.25)
Median [Min, Max]	76.0 [52.0, 89.0]	76.0 [56.0, 88.0]	77.5 [51.0, 88.0]	77.0 [51.0, 89.0]
RACE				
Black or African American	8 (9.3%)	9 (10.7%)	6 (7.1%)	23 (9.1%)
White	78 (90.7%)	74 (88.1%)	78 (92.9%)	230 (90.6%)
American Indian or Alaska Native	0 (0%)	1 (1.2%)	0 (0%)	1 (0.4%)

[

Table 1. Baseline Characteristics by Treatment]

Table 1. Baseline Characteristics by Treatment

Planned Treatment for Period 01	N	mean_age	sd_age	pct_female
Placebo	86	75.2	8.6	61.6
Xanomeline High Dose	84	74.4	7.9	47.6
Xanomeline Low Dose	84	75.7	8.3	59.5

Listing: First 20 Subjects | Listing: First 20 Subjects

Unique Subject Identifier	Planned Treatment for Period 01	Age	Sex
01-701-1015	Placebo	63	F
01-701-1023	Placebo	64	M
01-701-1028	Xanomeline High Dose	71	M
01-701-1033	Xanomeline Low Dose	74	M
01-701-1034	Xanomeline High Dose	77	F
01-701-1047	Placebo	85	F
01-701-1097	Xanomeline Low Dose	68	M
01-701-1111	Xanomeline Low Dose	81	F
01-701-1115	Xanomeline Low Dose	84	M
01-701-1118	Placebo	52	M
01-701-1130	Placebo	84	M
01-701-1133	Xanomeline High Dose	81	F
01-701-1146	Xanomeline High Dose	75	F
01-701-1148	Xanomeline High Dose	57	M
01-701-1153	Placebo	79	F
01-701-1180	Xanomeline High Dose	56	M
01-701-1181	Xanomeline High Dose	79	F
01-701-1188	Xanomeline Low Dose	71	M
01-701-1192	Xanomeline Low Dose	80	F
01-701-1203	Placebo	81	F

68.2 Listing: Subject-Level Listing

The code below transfer the output into a dataframe that only contains ASCII characters recommended by regulatory agencies. `tbl_base` is used as input for `r2rtf` to create the final report.

```
#> # A tibble: 11 x 5
#>   ` `      Placebo      `Xanomeline High Dose` `Xanomeline Low Dose` Overall
#>   <chr>      <chr>      <chr>      <chr>      <chr>
#> 1 ""        "(N=86)"      "(N=84)"      "(N=84)"      "(N=254)"
#> 2 "SEX"      ""          ""          ""          ""
#> 3 " Female"  "53 (61.6%)" "40 (47.6%)"  "50 (59.5%)"  "143 (56~
#> 4 " Male"   "33 (38.4%)" "44 (52.4%)"  "34 (40.5%)"  "111 (43~
#> # i 7 more rows
```

We define the format of the output. We highlight items that are not discussed in previous discussion.

`text_indent_first` and `text_indent_left` are used to control the indent space of text. They are helpful when you need to control the white space of a long phrase, “AMERICAN INDIAN OR ALASKA NATIVE” in the table provides an example.

In conclusion, the procedure to generate demographic and baseline characteristics table is summarized as follows:

- Step 1: Read the data set.
- Step 2: Use `table1::table1()` to get the baseline characteristics table.
- Step 3: Transfer the output from Step 2 into a data frame that only contains ASCII characters.
- Step 4: Define the format of the RTF table by using the R package `r2rtf`.

Exercises 1. Format Table 1 to N (`mean ± SD`) for age. 2. Add risk table to the KM plot (use an extension like `survminer` outside of this minimal example). 3. Create a listing that includes population flags once you derive them.

69 Capstone: End-to-End Mini Workflow

This chapter ties everything together: `read data` → `derive ADSL` → `produce TLFs` → `render a report`.

69.1 Parameters

69.2 1) Read (or Synthesize) SDTM

Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

`filter`, `lag`

The following objects are masked from 'package:base':

`intersect`, `setdiff`, `setequal`, `union`

Attaching package: 'lubridate'

The following objects are masked from 'package:base':

`date`, `intersect`, `setdiff`, `union`

[					
Table	1.	Baseline	by	Treatment	]
Table	1.	Baseline	by	Treatment	
Description of Planned Arm		N	mean_age	sd_age	pct_female
Placebo		226	75.04867	8.503715	60.61947
Screen Failure		52	75.09615	9.699928	69.23077
Xanomeline High Dose		184	74.01087	7.939656	48.36957
Xanomeline Low Dose		181	75.29834	8.277778	60.77348

69.3 2) Derive ADSL (Minimal Demo)

69.4 3) TLFs

69.5 4) Save Outputs

Challenge: Convert this chapter into a **parameterized report** (e.g., treatment subset or different cohort) and render multiple outputs.

70 Appendix: Tips, Profiles, .libPaths

70.1 Useful Profiles

Create ~/.Rprofile to set options (be careful on shared systems):

```
options(  
  repos = c(CRAN = "https://cloud.r-project.org"),  
  scipen = 999  
)
```

70.2 Custom Library Paths

```
# In .Rprofile or project-level .Rprofile  
.libPaths(c("/path/to/Rlibs", .libPaths()))
```

70.3 Format vs formatC (quick recap)

```
[1] "1.235e+02" "1.235e-03"
```

```
[1] "1.23456e+02" "1.23456e-03"
```

```
[1] "123.456" "0.001"
```

70.4 POSIXct vs POSIXlt

- **POSIXct**: seconds since epoch (numeric), compact, fast.
- **POSIXlt**: list-like with components (year, mon, mday...), easier to extract parts.

70.5 Recommended Packages

- `tidyverse`, `lubridate`, `janitor`, `gt`, `gtsummary`, `survival`, `broom`, `here`.
- Pharma/CDISC: `admiral`, `tlf/tern`, `pharmaverse` meta-packages (explore as you grow).

70.6 Short Glossary

- **SDTM**: Study Data Tabulation Model (FDA submission standard for raw domains).
- **ADaM**: Analysis Data Model (derived analysis-ready datasets).
- **TLF**: Tables, Listings, Figures for reporting.